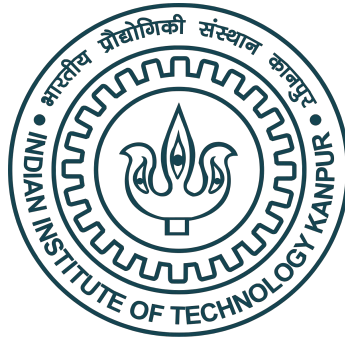

Safe and Robust Autonomous Landing of a Quadrotor on an Emulated Ship Deck: Barrier Function-based Model Predictive Control and Reinforcement Learning Approaches

*A Thesis Submitted
In Partial Fulfilment of the Requirements
for the Degree of BT - MT (Dual)*

Ritwik Shankar

(Roll No.: 218070866)

BT-MT Dual Degree

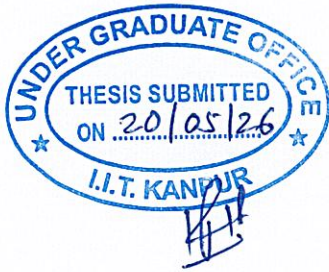


Department of Aerospace Engineering
Indian Institute of Technology Kanpur.

May, 2026

Certificate

It is certified that the work contained in the thesis titled **Safe and Robust Autonomous Landing of a Quadrotor on an Emulated Ship Deck: Barrier Function-based Model Predictive Control and Reinforcement Learning Approaches**, by **Ritwik Shankar** (Roll No. 218070866) has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.



May, 2026

Prof. Abhishek
Dept. of Aerospace Engineering,
Indian Institute of Technology Kanpur.

Prof. S.R. Sahoo
Dept. of Electrical Engineering,
Indian Institute of Technology Kanpur.

Declaration

This is to certify that the thesis titled **Safe and Robust Autonomous Landing of a Quadrotor on an Emulated Ship Deck: Barrier Function-based Model Predictive Control and Reinforcement Learning Approaches** has been authored by me. It presents the research conducted by me under the supervision of **Prof. Abhishek** and **Prof. S.R. Sahoo**.

To the best of my knowledge, it is an original work, both in terms of research content and narrative, and has not been submitted elsewhere, in part or in full, for a degree. Further, due credit has been attributed to the relevant state-of-the-art and collaborations with appropriate citations and acknowledgments, in line with established norms and practices.



Ritwik Shankar

Roll No. 218070866

BT-MT

Dept. of Aerospace Engineering,
Indian Institute of Technology Kanpur,
Kanpur, 208016, India.

Synopsis

Name of the Student: Ritwik Shankar

Roll No: 218070866

Degree for which submitted: BT-MT
Engineering

Department: Dept. of Aerospace

Thesis Title: Safe and Robust Autonomous Landing of a Quadrotor on an Emulated Ship Deck: Barrier Function-based Model Predictive Control and Reinforcement Learning Approaches

Thesis Supervisor: Prof. Abhishek, Prof. S.R. Sahoo

Month and Year of Thesis Submission: May 2026

Autonomous landing of a quadrotor on a moving ship deck is a tightly constrained control problem of direct relevance to maritime logistics and naval recovery operations. The difficulty arises because the landing surface is not fixed in either position or orientation, while the aerial vehicle must still achieve deck-relative tracking, choose a safe descent instant, and satisfy strict touchdown constraints on attitude and relative velocity. This thesis studies this problem through two complementary routes: first, a prediction-driven safe-landing controller for a heaving deck, and then a reinforcement-learning workflow compatible with practical PX4-style deployment.

The central technical contribution of the thesis is the heave-landing controller framework, where the dominant difficulty lies directly along the touchdown axis. In this setting, touchdown must be synchronised with the vertical motion of the deck. To address this, the thesis develops a GRU-based prediction module for short-horizon deck-heave forecasting, a receding-horizon vertical trajectory generator posed as a constrained optimisation problem, and a barrier-based safety filter that prevents unsafe commitment when the available stopping margin becomes infeasible. The model-predictive control formulation plans descent toward a selected future deck-heave peak while respecting state and actuation limits, and the control barrier function enforces a stage-zero safety condition based on relative distance, relative velocity, and stopping distance. Vision-driven hardware experiments show that the resulting landing pipeline can recover from unfavourable touchdown windows through abort-and-retry behaviour. In the final vision-based heave experiments, all 11 out of 11 tests ended in successful landing, ranging from relative touchdown speed as high as 0.31 m/s, including cases in which the controller had to abandon an unsafe descent and attempt landing again at a later deck-motion phase.

The latter part of the thesis develops an open-source reinforcement-learning training and transfer workflow for UAV landing that preserves a practical velocity-and-yaw-rate command interface. The workflow combines high-throughput vectorised pretraining with a PX4-like cascaded controller backend and a lower-throughput fine-tuning stage against PX4 SITL, ensuring the learned policy remains compatible with real flight-stacks. This stack is then specialised into two tasks: sway landing and heave landing. The sway task learns deck-relative tracking, descent regulation,

and touchdown behaviour in a vectorised environment, while the heave extension introduces ship-data-driven platform motion, privileged critic information, and recurrent GRU-based actor–critic policies to handle the stronger temporal structure of vertical deck motion. The final sway policy achieves 93.96% touchdown terminations in training-simulation, with 91.23% good touchdowns over all episodes and 96.72% good touchdowns among touchdown episodes. The recurrent heave RL policy reaches 98.61% touchdown terminations, with 91.25% good touchdowns over all episodes and 92.47% good touchdowns among touchdown episodes. The same deployment-oriented stack is also used in ground experiments with an ArduPilot-based vehicle to land on a moving ship-deck emulator, demonstrating that the learned policy can be evaluated through the same velocity-command interface outside the training simulator.

Taken together, the thesis develops a unified view of autonomous ship-deck landing across heuristic control, prediction-based safe landing, and deployment-oriented reinforcement learning. Its main outcome is not only a set of landing controllers, but a complete experimental and algorithmic workflow that connects deck sensing, touchdown timing, safety filtering, and trainable policy-based landing under a command interface that remains meaningful for practical flight controllers.

Acknowledgements

This report is not only the result of the author's continuous effort, but also of many people involved in the background. I want to start by expressing my gratitude to the staff at Helicopter Lab for making it very easy for me to focus on my research and leave trivial matters to them. No amount of words could express my gratitude to Prof. Abhishek. It's only his guidance and continuous support over the last three years that have enabled me to excel academically, in research and personally. I also want to thank Prof. S.R. Sahoo and Prof. K.S. Venkatesh for their guidance throughout the respective projects. I want to especially thank Mr Chiranjeev Prachand and Mr Jitendra Singh for guiding me from the moment I had very little idea of what I was doing. It's difficult to imagine this work's success without their support and contribution. I also thank Prof. Sandip Tiwari for letting me have some of his infinite wisdom, both in academics and in life.

I want to thank IITK FC for helping me blow off steam after a tough day, and always pushing me to my best. I also want to express my sincere gratitude to the people very close to me, Mridul, Tushar, Sid, Deekshansh, and Akku. At last, I owe everything to my mom, Neetu, my dad, Shiv, my little sister, Deepika, and BumbleBee; it's they who helped me become the person I am today, and for that, I could never repay this debt of gratitude.

Ritwik

Contents

Acknowledgements	v
List of Figures	viii
List of Tables	ix
Abbreviations	x
Symbols	xii
List of Publications	xii

1 Introduction	1
1.1 Motivation	1
1.2 Literature Review	2
1.2.1 Vision-Based Ship-Deck Landing and Moving-Platform Studies	2
1.2.2 Heave Landing, Motion Estimation, and Prediction	4
1.2.3 Deck Emulation and Experimental Validation	5
1.3 Research Gap	5
1.4 Contributions	6
1.5 Organisation of the Thesis	7
1.6 Summary	8
2 Controller Design for Safe Landing on a Heaving Ship-Deck Emulator	9
2.1 Introduction	9
2.2 Heave-Dominant Landing Problem and System Architecture	10
2.2.1 Overall System Architecture	11
2.2.2 Relative Vertical Landing Formulation	13
2.3 Prediction Model	13
2.3.1 Dataset and Design Constraints	14
2.3.2 Statistical and Convolutional Baselines	14

2.3.2.1	ARIMA Baseline	14
2.3.2.2	Temporal Convolutional Network	15
2.3.2.3	Minor Component Analysis	16
2.3.3	Recurrent Predictors for Heave Forecasting	19
2.3.3.1	Shared Training Objective for LSTM and GRU	19
2.3.3.2	LSTM Predictor Family	22
2.3.3.3	GRU Predictor Family	24
2.3.3.4	Results from the GRU Sweep	26
2.3.4	Final Predictor and Peak-Target Selection	28
2.3.4.1	Final Motion Predictor model	28
2.3.4.2	Peak-Target Selection	28
2.4	Vertical Trajectory Generator	29
2.4.1	Discrete Vertical Dynamics and Reference Construction	30
2.4.2	Receding-Horizon Landing Objective	31
2.5	Safe Landing Logic and Safety Filter	34
2.5.1	Control Barrier Function for Vertical Landing Safety	35
2.5.2	Operational Landing State Machine and Fallback Logic	39
2.6	Results and Discussion	41
2.6.1	Simulation Validation of the MPC and CBF	41
2.6.2	Hardware Demonstration of the landing pipeline	44
2.7	Summary	45
3	Development of a PX4-compatible Reinforcement Learning Training Stack	51
3.1	Introduction	51
3.2	Why Reinforcement Learning for Aerial Robotics	52
3.2.1	Limitations of Classical Controllers in Complex Landing Tasks	52
3.2.2	Why RL Has Become Important in Robotics	53
3.2.3	High-Performance RL Exists, but Reusable Flight-Stack-Compatible Pipelines Remain Scarce	54
3.3	Practical Gap and Scope of This Work	55
3.4	Design Objectives and System Requirements	56
3.5	Stage 1: Vectorised Pretraining in a PX4-Like Environment	58
3.5.1	Motivation for the Pretraining Stage	58
3.5.2	Environment Contract Used During Pretraining	58
3.5.3	Embedded PX4-Like Control Pipeline	59
3.5.4	Why Full SITL Is Not Used in Stage 1	64
3.6	Stage 2: PX4 SITL Fine-Tuning	64
3.7	Transfer Validation and Deployment Pipeline	65
3.8	Summary	66
4	Landing on a Ship-Deck Emulator Using RL	68

4.1	Part I: Sway-Landing Task	68
4.1.1	Task Definition and Simulation Setup	69
4.1.2	Vehicle and Platform Models	71
4.1.3	Observation Space	73
4.1.4	Action Space and Command Semantics	75
4.1.5	Reward Design for Sway Landing	75
4.1.6	Termination Logic, Reset Distributions, and Domain Ran- domisation	78
4.1.7	PPO Configuration for the Base and Fine-Tuned Sway Policies	80
4.1.8	Sway-Landing Results	82
4.1.8.1	Hardware Results	82
4.1.9	Remarks	83
4.2	Part II: Heave-Landing Extension	83
4.2.1	Why Heave Landing is a Structured Extension of the Sway Task	84
4.2.2	Heave dataset for RL	84
4.2.3	Actor Observation and Privileged Critic Observation	85
4.2.4	Reward and Touchdown Changes Relative to Sway Landing .	86
4.2.5	Privileged-Critic Recurrent GRU Policies	88
4.2.6	Heave-Landing Results	90
4.2.7	Summary of the RL Landing Chapter	91

List of Figures

2.1	UAV used for all Heave Landing Experiments	11
2.2	Stewart-like platform with four degrees of freedom used to emulate ship-deck motion.	12
2.3	Overall architecture of the predictive and safe heave-landing framework.	12
2.4	Snapshot of ARIMA-based predictions	15
2.5	Snapshot of TCN-based predictions	16
2.6	Snapshot of MCA-based predictions	19
2.7	Qualitative comparison of GRU predictors from the heave-model sweep.	27
2.8	Visualisation of CBF parameters	36
2.9	Gazebo simulation environment with the Iris UAV and the Stewart-platform-based moving deck.	43
2.10	Simulation validation of the vertical landing pipeline. The top row shows the deliberately biased touchdown-target case and the bottom row shows a successful GRU-guided landing with a touchdown velocity of -0.22 m/s.	46
2.11	Experiment Setup with the Ship Deck Emulator	47
2.12	Vision-based landing, Case 4.	48
2.13	Vision-based landing, Case 11.	49
2.14	Landing sequence for Case 4.	50
3.1	Overall two-stage RL training and transfer stack implemented in this work.	57
3.2	PX4-like cascaded controller used during vectorized pretraining, reproduced from the control/data flow used in the <code>uav_rl</code> repository.	63
3.3	Deployment-side execution pipeline used after policy training.	66
4.1	Moving landing platform used in the RL landing tasks.	69
4.2	Iris UAV configuration with landing legs used during simulation training.	70
4.3	Parallel vectorised training of the sway-landing task in Isaac Lab.	71
4.4	Isaac Lab environment used for the RL landing	72
4.5	RL landing rollout on sinusoidal deck motion with amplitude 0.45 m and frequency 0.1 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 25$ s.	92

4.6	RL landing rollout on sinusoidal deck motion with amplitude 0.45 m and frequency 0.2 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 17$ s.	93
4.7	RL landing rollout on sinusoidal deck motion with amplitude 0.55 m and frequency 0.1 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 33$ s.	94
4.8	RL landing rollout on sinusoidal deck motion with amplitude 0.55 m and frequency 0.2 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 24$ s.	95
4.9	RL landing snapshots on sinusoidal deck motion with amplitude 0.55 m and frequency 0.2 Hz.	96

List of Tables

2.1	Best LSTM configurations from the current heave-prediction log sweep. Each row reports the best completed run retained for that architecture.	24
2.2	Best GRU configurations from the current heave-prediction log sweep. Each row reports the best completed run retained for that architecture.	27
2.3	Vertical MPC configuration used in the heave-landing controller. . . .	34
2.4	Barrier-function and supervisory parameters used in the safe-landing filter.	39
3.1	Main components of the proposed RL stack.	58
4.1	Policy observation vector used in the baseline sway policy.	73
4.2	Grouped active reward components in the base sway policy.	76
4.3	Active domain-randomisation settings in the sway-landing task. . . .	80
4.4	PPO settings for the sway-landing policy and its continuation fine- tuning run.	81
4.5	Final simulation metrics for the fine-tuned sway policy.	82
4.6	Actor and critic observation structure in heave_landing	86
4.7	Grouped active reward components in the recurrent heave policy. . .	87
4.8	PPO runner configurations defined for heave_landing	89
4.9	Final simulation metrics for the recurrent heave policy.	90

Abbreviations

ARIMA	A utoregressive I ntegrated M oving A verage
BiLSTM	B idirectional L ong S hort- T erm M emory
CBF	C ontrol B arrier F unction
CFD	C omputational F luid D ynamics
CPU	C entral P rocessing U nit
GPS	G lobal P ositioning S ystem
GPU	G raphics P rocessing U nit
GRU	G ated R ecurrent U nit
HIL	H ardware- I n-the- L oop
HPIPM	H igh- P erformance I nterior P oint M ethod
IMU	I nertial M easurement U nit
IR	I nfrared
KL	K ullback– L eibler
LiDAR	L ight D etection and R anging
LSTM	L ong S hort- T erm M emory
MAVLink	M icro A ir V ehicle L ink
MCA	M inor C omponent A nalysis
MLP	M ulti- L ayer P erceptron
MoCap	M otion C apture
MPC	M odel P redictive C ontrol
MSE	M ean S quared E rror
OFFBOARD	E xternal high-level flight-command mode in PX4
PID	P roportional– I ntegral– D erivative
PPO	P roximal P olicy O ptimization
PSO	P article S warm O ptimization
PX4	O pen-source autopilot framework used for multicopter control
RAM	R andom A ccess M emory

RL	R einforcement L earning
RTK-GPS	R ead- T ime K inematic G lobal P ositioning S ystem
SCONE	S ystematic C haracterization of the N aval E nvironment
SITL	S oftware- I n-the- L oop
SQP	S equential Q uadratic P rogramming
TCN	T emporal C onvolutional N etwork
UAV	U nmanned A erial V ehicle
USV	U nmanned S urface V ehicle
VTOL	V ertical T akeoff and L anding

Symbols

A	Rotor-allocation mapping thrust and torques to rotor-speed commands
a_p	Platform vertical acceleration in the heave-landing model
$a_{\text{rel,max}}$	Assumed relative braking acceleration used in safety filter
B	Matrix of retained minor-component eigenvectors in the MCA predictor
B_1, B_2	MCA eigenvector matrix
$\mathcal{C}$	Safe set enforced by the control barrier function
C_{drag}	Body-drag coefficient vector in the PX4-like motor model
d	Relative vertical separation between the UAV and the deck
d_{env}	Inter-environment spacing used in vectorized RL training
d_{min}	Minimum allowed vertical clearance in the heave-landing safety filter
d_{stop}	Estimated stopping distance required before safe touchdown
$\delta_{\text{cbf},k}$	Stage parameter that activates the barrier term in the heave optimizer
Δt	Discrete-time sample interval
$\Delta t_{\text{sim}}, \Delta t_{\pi}$	Physics and policy update periods in the RL landing environments
$\mathbf{e}_{\omega}$	Body-rate tracking error in the PX4-like rate controller
$\mathbf{e}_R$	Attitude error used by the PX4-like attitude controller
$\mathbf{e}_v$	Velocity-tracking error used by the PX4-like velocity controller
$\mathbf{F}_{\text{drag}}$	Aerodynamic drag force applied in the PX4-like motor model
F_i	Rotor thrust generated by the i -th propeller
$\mathbf{F}_{\text{sp}}$	Force setpoint generated from acceleration and gravity compensation
F_{th}	Contact-force threshold used for touchdown detection in RL landing
g	Gravitational acceleration magnitude
$\mathbf{g}_b$	Gravity direction projected into the UAV body frame
γ	Linear class- $\mathcal{K}$ gain used in the control barrier function condition
h	Barrier-function value defining the safe heave-landing set
H	Prediction horizon length

h_0	Stage-zero value of the heave-landing safety barrier
$\mathcal{L}_{\text{pos}}$	Position prediction-loss components for recurrent model training
$\mathcal{L}_{\text{jump}}$	Jump prediction-loss components for recurrent model training
$\mathcal{L}_{\text{vel}}$	Velocity prediction-loss components for recurrent model training
$\mathcal{L}_{\text{acc}}$	Acceleration prediction-loss components for recurrent model training
λ_{jump}	Weight on jump continuity loss in recurrent training
λ_{vel}	Weights on velocity continuity loss in recurrent training
λ_{acc}	Weights on acceleration continuity loss in recurrent training
N	Number of control stages in MPC, equal to $H - 1$
n	Past-history length in the MCA window definition
N_{env}	Number of parallel RL environments used during vectorized training
$N_{\text{in}}, N_{\text{out}}$	Input-history and forecast lengths for the recurrent predictor
ω	UAV body angular-velocity vector in the PX4-like rate controller
ω_i	Angular speed of the i -th rotor
ω_{rel}	Relative angular-velocity vector between the UAV and the deck
ω_{sp}	Body-rate setpoint generated by the PX4-like attitude loop
P	Number of retained minor components in the MCA predictor
p_k	Stage-parameter vector supplied to the heave optimizer at step k
$\mathbf{p}_{\text{plat},0}$	Initial platform position in the RL landing environment
$\mathbf{p}_{\text{rel}}$	Relative position vector between the UAV and the deck
$\tilde{\mathbf{p}}_{\text{rel}}$	Relative position vector with touchdown-offset correction
ϕ, θ, ψ	Roll, pitch, and yaw angles
$\mathbf{q}_{\text{sp}}$	Desired attitude quaternion generated by the PX4-like controller
$\mathbf{q}_{\text{uav}}^w$	UAV root quaternion expressed in the world frame
R_{bad}	Penalty assigned to an unsafe or poor touchdown
R_{center}	Center-proximity touchdown bonus used in the RL landing reward
r_{clr}	Vertical-clearance component of the RL landing reward
r_{track}	Position-tracking component of the RL landing reward
r_{vel}	Velocity-tracking component of the RL landing reward
R_{good}	Reward assigned to a good touchdown
R_{sp}	Desired rotation matrix in the PX4-like attitude controller
σ_{xy}, σ_v	Position and velocity shaping scales used in the RL landing reward
T_{ep}	Episode duration in the RL landing environments
t_{hover}	Initial hover time in the heave-landing supervisor
t_{now}	Initial current time in the heave-landing supervisor

t_{peak}	Initial selected future peak time in the heave-landing supervisor
T_p	Forecast horizon duration for deck-motion prediction
T_{return}	Hover-wait duration before retrying heave landing after an abort
T_{sp}	Collective thrust setpoint in the PX4-like controller
τ_{sp}	Torque setpoint generated by the PX4-like rate loop
$\mathcal{U}$	Admissible control set in the general control-affine CBF formulation
$\mathbf{u}$	Generic policy or controller action vector
u_k	Vertical acceleration command at stage k in the heave-landing optimizer
$u_{\min}, u_{\max}$	Minimum and maximum commanded vertical accelerations in MPC
$\mathbf{u}_{\text{raw}}, \mathbf{u}_{\text{sp}}$	Raw policy command setpoint in the PX4-compatible RL stack
u_x, u_y, u_z	Control-output velocity in the x , y , and z directions
v_L	Maximum admissible touchdown relative-speed magnitude in CBF
$v_{\min}, v_{\max}$	Minimum and maximum velocity clipping limits
v_p, v_u	Platform and UAV vertical velocities in the heave-landing model
v_r	Relative vertical velocity between the UAV and the deck
$\mathbf{v}_{\text{rel}}$	Relative linear-velocity vector between the UAV and the deck
$\mathbf{v}_{\text{sp}}$	Velocity setpoint issued to the PX4-like controller
w_u	Optimisation weight on control effort
w_v	Optimisation weight on velocity regularisation
x, x_k	State vectors used in the CBF model and MPC
$\hat{\mathbf{y}}, \mathbf{y}$	Predicted and true future heave sequences in recurrent-model training
z_0	Vertical offset between the UAV body origin and the touchdown point
z_{curr}	Current UAV altitude
z_d	Deck altitude in the heave-landing model
z_{hover}	Hover altitude used before landing or after abort-and-retry recovery
z_k	Predicted altitude at stage k of the heave-landing optimizer
z_{peak}	Predicted deck-height peak selected as the touchdown target
z_u	UAV altitude in the heave-landing model

List of Publications

Articles Published in International Conferences/Journals

1. **R. Shankar**, C. Prachand, J. Singh, A. Abhishek, K.S. Venkatesh, “Vision-Based Landing of UAV on Simulated Ship Deck with Roll Pitch and Sway Motions”, 2025 Vertical Flight Society, Forum 81
2. **R. Shankar***, C. Prachand*, A. Abhishek, K.S. Venkatesh, “Autonomous Quadrotor Landing on a Heaving Ship Deck Using Prediction, Receding-Horizon Optimisation, and Safety Filtering”, Manuscript under preparation for T-RO
3. **R. Shankar**, A. Abhishek, S.R. Sahoo, "Reinforcement Learning-Based Autonomous Landing on Moving Ship Decks", Manuscript under preparation for ICRA 2027
4. C. Prachand, R. Rustagi, **R. Shankar**, J. Singh, A. Abhishek, K.S. Venkatesh, "Vision-Based Autonomous Ship Deck landing of an Unmanned Aerial Vehicle using Fractal ArUco markers", 2025 AIAA SciTech Forum

Dedicated to papa ji

Chapter 1

Introduction

1.1 Motivation

Autonomous landing of a quadrotor on a moving ship deck is a tightly constrained control problem with direct relevance to maritime logistics, naval recovery operations, and long-endurance autonomous deployment. The deck is not fixed in either position or orientation, while the aerial vehicle must still achieve deck-relative tracking, regulate its attitude, and satisfy strict touchdown constraints on relative velocity and contact timing. Among the different deck-motion components, heave is especially critical because it acts directly along the touchdown axis. A controller may achieve good lateral alignment and still fail if the platform is moving upward at contact or if the available stopping distance collapses too quickly near the deck.

This thesis therefore begins from the vertical-landing problem rather than from generic moving-platform tracking. The central controller-design question is how to combine short-horizon deck-motion prediction, receding-horizon descent planning, and an explicit safety filter so that the vehicle descends only when the future touchdown condition remains feasible. The emphasis is not only on simulation, but on a

full experimental pipeline in which the controller is tested on a heaving ship-deck emulator using onboard sensing and hardware-oriented decision logic.

The later chapters then build on that foundation. Once the structure of a safe heave-landing controller is established, the thesis turns to reinforcement learning as a complementary route for moving-deck landing, first by constructing a PX4-compatible training stack and then by instantiating that stack on sway-landing and heave-landing RL tasks.

1.2 Literature Review

1.2.1 Vision-Based Ship-Deck Landing and Moving-Platform Studies

Autonomous landing of vertical takeoff and landing UAVs has been studied using sensing modalities such as IR-LEDs, LiDAR, RTK-GPS, and onboard vision [1]. Broad reviews such as [2] show that, for moving-platform landing, vision remains especially attractive because it provides deck-relative pose information with lightweight onboard hardware and without requiring a high-cost localisation system on the landing target.

Early visual landing methods relied on geometric landing patterns and engineered deck targets. Xu et al. [3] studied a T-shaped shipboard landing target, Zeng et al. [4] used H-shaped markers in a vision-based UAV landing system, and Benini et al. [5] used concentric circles for real-time pose estimation during autonomous takeoff and landing. Fiducial-marker systems improved robustness relative to these simpler patterns. Fiala [6] compared ARTag and ARToolkit Plus, Zhang et al. [7] used

fiducial-marker pose estimation for precise quadrotor landing, Wang and Olson [8] introduced AprilTag 2 as a faster and more robust detector, and Romero-Ramirez et al. [9] developed a sped-up ArUco detector that has since become standard in robotic landing experiments. Bhargavapuri et al. [10] demonstrated marker-based landing on a moving vehicle, while Kawamura et al. [11] explored IMU-aided vision for precision approach and landing.

Ship-deck landing adds the further difficulty that the target itself moves with coupled deck dynamics. Venugopalan et al. [12] reported an early demonstration of autonomous landing on a small marine vehicle. Lin et al. [13] proposed a hybrid visual marker for cluttered shipboard environments, Wang and Bai [14] combined LEDs with H-shaped pattern detection for vessel-deck landing, and Sanchez-Lopez et al. [15] developed a vision-based shipboard landing system on a six-servo platform. Polvara et al. [16] studied landing on a perturbed USV deck in simulation, Shastry et al. [17] considered vision-based landing on a stochastically moving platform, and Lee et al. [18] reported intelligent vision-based autonomous ship landing of VTOL UAVs.

The studies most relevant to the present thesis are those that combine realistic deck motion with experimentally validated perception and control. Prachand et al. [19] showed that fractal ArUco markers can sustain deck-relative pose estimation over a wide altitude range without the bookkeeping overhead of independent multi-marker layouts. Gupta et al. [20] addressed UAV landing on a USV in harsh winds and turbulent open water using MPC and online motion decomposition to target near-zero deck tilt, while Procházka et al. [21] developed an MPC-based trajectory generator for agile landing on a moving boat with continuously updated deck-state predictions.

1.2.2 Heave Landing, Motion Estimation, and Prediction

Heave-dominant landing is more specific than general moving-deck tracking because the dominant difficulty lies directly along the touchdown axis. In this regime, good lateral alignment is not enough; the vehicle must also synchronise its descent with the deck's vertical motion. Hu et al. [22] addressed this through a control architecture composed of motion estimation, time-optimal trajectory generation, and tracking control for landing on a vertically moving platform. Nguyen et al. [23] later proposed a precision-landing method for a heaving platform that combines robust altitude regulation under ground effect and disturbances with a landing-target state estimator and a dedicated landing procedure.

This heave-landing literature is supported by a parallel body of work on ship-motion estimation. Triantafyllou, Bodson, and Athans [24] used Kalman filtering to estimate heave, pitch, roll, sway, and yaw for VTOL landing applications, explicitly tying ship-motion estimation to safe aerial recovery. Richter et al. [25] developed an adaptive IMU-based heave estimator that reduces phase error through online parameter adaptation, which is important whenever the estimated vertical motion is used inside a real-time control loop.

The need to predict ship motion before touchdown has also long been recognised. Sidar and Doolin [26] investigated whether aircraft-carrier pitch and heave could be predicted in real time and showed that useful prediction horizons of several seconds are achievable with Kalman-filter-based prediction, including accurate zero-crossing estimates. More recent work has expanded on this idea using data-driven predictors. Keijzer [27] studied ship-heave prediction using LSTM networks without wave-excitation measurements and showed that useful prediction quality can still be obtained from motion history alone, especially over shorter horizons. Geng et al. [28]

proposed a short-term ship-motion prediction method that combines empirical mode decomposition with an adaptive PSO–LSTM predictor in a sliding-window framework to handle nonlinear and nonstationary motion data. Yang et al. [29] compared reservoir computing with recurrent baselines such as LSTM, BiLSTM, and GRU for real-time ship-motion prediction in irregular waves, showing that lightweight sequence models can be competitive for online maritime forecasting.

1.2.3 Deck Emulation and Experimental Validation

Experimental validation of ship-deck landing usually depends on Stewart platforms or Stewart-like parallel manipulators. Their kinematics and dynamics have been studied extensively in the literature, including general kinematic formulations [30], dynamic analysis [31, 32], and laboratory-scale ship-deck emulation for UAV landing [19, 33]. These systems matter because the landing controller must be tested on repeatable deck motion before claims about prediction, timing, and safety can be taken seriously.

At the same time, laboratory emulators often involve a tradeoff between realism, cost, and motion authority. A useful experimental platform must be repeatable enough for comparison, but also rich enough to expose failure modes in heave-dominant touchdown. This motivates the use of a heaving ship-deck emulator as the primary validation environment for the controller designed in this thesis.

1.3 Research Gap

The literature establishes that autonomous landing on moving platforms is feasible, but the heave-dominant landing problem still remains incomplete in three important

ways.

First, there is an integration gap between deck sensing, motion prediction, and landing execution. Many published systems demonstrate some combination of relative-pose estimation, tracking, or touchdown logic, but fewer works show a complete pipeline in which future deck motion is explicitly forecast and then used online to select a safer touchdown opportunity.

Second, there is a control-design gap in how prediction is converted into descent action. A forecast alone is not enough; the vehicle still needs a vertical landing controller that respects state and actuation limits, synchronises with the predicted deck peak, and remains operational when the future trajectory no longer supports safe touchdown. This motivates a receding-horizon descent planner together with a supervisory landing logic.

Third, there is a safety gap near touchdown. Even with a prediction-driven descent planner, the vehicle may still reach a condition in which the remaining stopping distance is insufficient for safe contact. The literature provides strong motivation for explicit safety filtering, but a practical landing controller still needs a barrier-based condition that can decide whether to continue descent or abort and retry.

1.4 Contributions

The controller-design part of the thesis makes the following contributions:

- a short-horizon GRU-based deck-motion prediction;

- a receding-horizon vertical landing optimiser that plans touchdown toward a selected future deck-heave peak while respecting state and actuation limits; and
- a control-barrier-based safety layer that prevents unsafe descent commitment and enables abort-and-retry behaviour when the available stopping margin becomes infeasible.

The later parts of the thesis then extend the overall study to reinforcement learning by developing a PX4-compatible training stack and by instantiating that stack on sway-landing and heave-landing RL tasks.

1.5 Organisation of the Thesis

The remainder of the thesis is organised as follows. Chapter 2 presents the controller design for safe landing on a heaving ship-deck emulator, including the system architecture, the prediction module, the receding-horizon descent planner, the barrier-based safety condition, and the experimental landing results. Chapter 3 then develops the PX4-compatible reinforcement-learning training stack, with emphasis on vectorised pretraining, SITL fine-tuning, and transfer validation. Finally, Chapter 4 instantiates that stack on ship-deck landing tasks, first through sway landing and then through a heave-landing RL extension with privileged critic information and recurrent policies.

1.6 Summary

This chapter introduced the ship-deck landing problem with emphasis on heave-dominant touchdown, reviewed the literature on deck-relative sensing, heave-motion estimation, short-horizon prediction, and experimental deck emulation, and identified the main control-design gaps that motivate the thesis. It then stated the contributions of the controller-design part of the work in terms of prediction, optimisation, and explicit safety filtering. The next chapter develops that heave-landing controller in detail and presents its experimental validation on a heaving ship-deck emulator.

Chapter 2

Controller Design for Safe Landing on a Heaving Ship-Deck Emulator

2.1 Introduction

Heave landing is treated separately from the general moving-platform problem because the dominant difficulty is aligned directly with the touchdown axis. A controller may achieve excellent lateral alignment and still fail catastrophically if it reaches the deck while the platform is moving upward or while the relative vertical speed remains too large. For that reason, the heave-landing pipeline in this thesis combines three elements: future deck-motion prediction, receding-horizon descent planning toward a favourable touchdown instant, and an explicit safety filter that prevents unsafe commitment when prediction or relative motion makes touchdown infeasible.

The chapter is organised around the vertical landing pipeline itself. It first formulates the heave-dominant landing problem and the associated aerial-deck system

architecture. It then studies the prediction block, beginning with statistical and convolutional baselines and moving to recurrent predictors, hybrid residual modelling, and the final peak-selection logic used to choose a touchdown opportunity. The next part derives the vertical trajectory generator from the discrete dynamics and the receding-horizon objective. The chapter then introduces the safety layer through a barrier-based landing filter and the supervisory state machine that governs landing, abort, and retry behaviour. It closes with hardware-oriented experimental results using MoCap and vision data.

Within the scope of the thesis, the heave-landing chapter makes three concrete technical contributions:

- a prediction module that forecasts future deck heave and converts that forecast into a candidate touchdown instant near a favourable deck-motion peak,
- a receding-horizon vertical landing optimiser that plans descent toward the selected touchdown opportunity while respecting state and actuation limits, and
- a barrier-based safety filter that prevents unsafe commitment when the predicted landing condition becomes infeasible.

2.2 Heave-Dominant Landing Problem and System Architecture

Heave motion is especially difficult because it changes the available touchdown window in the same direction as the landing manoeuvre itself. Even when horizontal alignment is good, poor synchronisation between descent and deck vertical motion

can lead to high relative impact speed, unstable contact, or missed landing opportunities. This makes the heave landing more timing-sensitive than planar deck tracking alone.

2.2.1 Overall System Architecture

The overall system contains an aerial subsystem (Fig. 2.1) and a ground subsystem (Fig. 2.2). The aerial subsystem includes the quadrotor, a downward-facing camera, a Cube Orange with an ArduPilot flight controller, and an onboard Jetson Nano for all computation. The ground subsystem consists of the ship-deck emulator instrumented with visual markers. The lateral tracking and vision-based relative pose estimation pipeline follows our earlier work [19, 33], which is a part of the PhD thesis of Mr Chiranjeev Prachand; in the present report, the relative position estimation is already attained using the previously cited work, and this report only focuses on the heave-prediction, vertical-planning, and safe-touchdown logic.



FIGURE 2.1: UAV used for all Heave Landing Experiments

Figure 2.3 shows the proposed vertical landing pipeline used in this chapter. A future deck-heave trajectory is predicted from recent motion history, a candidate future deck-heave maximum (or peak) is selected as the touchdown target, and a

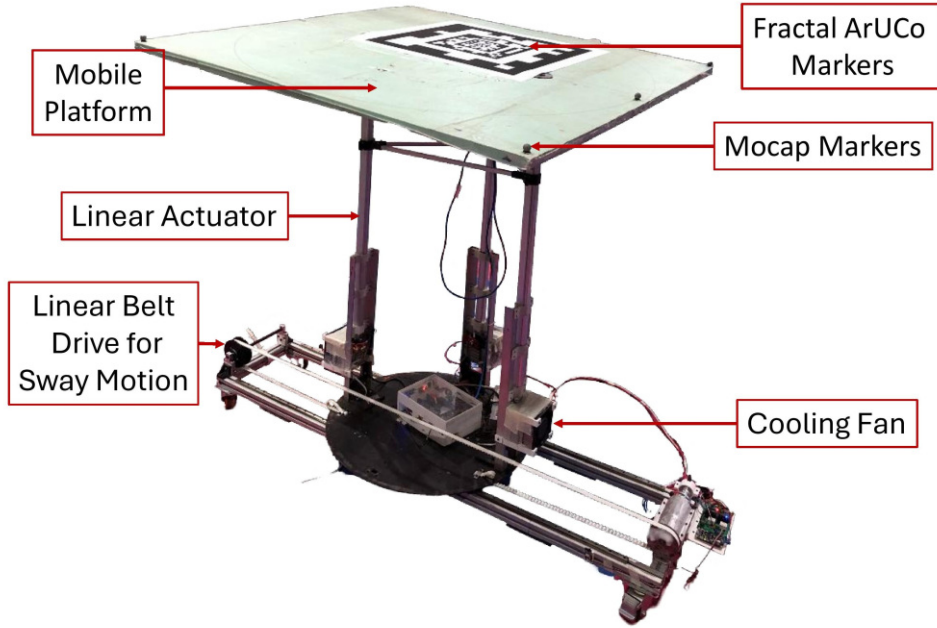


FIGURE 2.2: Stewart-like platform with four degrees of freedom used to emulate ship-deck motion.

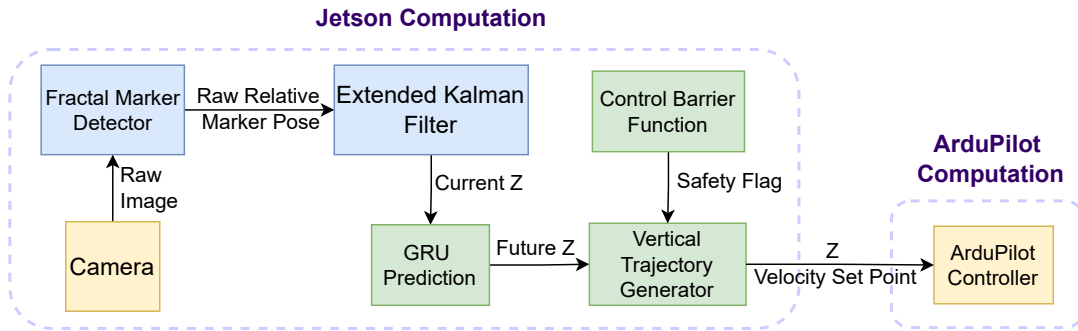


FIGURE 2.3: Overall architecture of the predictive and safe heave-landing framework.

short-horizon optimiser generates a vertical descent profile. A safety layer evaluates whether the current relative state can still be brought to touchdown safely and intervenes whenever the predicted descent would violate the available braking margin.

2.2.2 Relative Vertical Landing Formulation

Let z_u denote the UAV altitude and z_d denote the deck altitude along the vertical axis. The relative vertical separation is

$$d = z_u - z_d. \quad (2.1)$$

A safe touchdown requires not only $d \rightarrow 0$, but also a sufficiently small relative vertical velocity near contact. To minimise the relative velocity at touchdown, we need an estimate of the platform's future motion, i.e., a timestamp at which the platform's velocity is negligible, simplifying the problem to just landing a UAV with near-zero velocity at that timestamp.

2.3 Prediction Model

The prediction block used for the heave landing must forecast the future deck altitude over a horizon long enough for the descent planner to decide not only whether the platform is currently safe to approach, but also whether a safer touchdown opportunity will occur a short time later. The scaled-experiment study reported by Penn State [34] was a useful starting point, because it framed deck-heave prediction as a forecasting problem directly coupled to landing timing. However, that study also made clear that an operational predictor for this thesis must satisfy three additional constraints: it must run comfortably inside a 20 Hz landing loop, it must remain accurate over the first few seconds of the prediction horizon where touchdown decisions are made, and it must tolerate a realistic level of noise in the sensed motion history rather than relying on laboratory-only motion-capture signals.

2.3.1 Dataset and Design Constraints

The prediction experiments were conducted on five heave time histories derived from the SCONE dataset [35]. Each record contained 36,000 samples at 20 Hz. Four datasets were used for training and one for held-out testing. The broader ship-motion study included both roll-dominant and heave-dominant sequences, identified in the original work as ‘SCONE_D3R_1’ and ‘SCONE_D3H_1’, but the prediction model in this chapter focuses specifically on the heave channel Z because reliable real-time deck-velocity measurements are generally not available in the field.

The predictor was screened against three operational constraints:

- inference time below 0.04 s so that the prediction can run comfortably inside a 20 Hz control loop,
- negligible error over the first three seconds of the forecast window, since this part of the horizon is most critical for touchdown timing, and
- reasonable robustness to mild outliers and non-sinusoidal disturbances in the test sequence.

2.3.2 Statistical and Convolutional Baselines

Several lower-complexity predictors were evaluated before the final recurrent design was chosen.

2.3.2.1 ARIMA Baseline

An initial statistical baseline was implemented using a nonseasonal ARIMA(p, d, q) model [36] through the `statsmodels` implementation. In this representation, p is

the number of autoregressive terms, d is the number of nonseasonal differences used to enforce stationarity, and q is the number of lagged forecast errors in the prediction equation. A model of order $(3, 1, 1)$ was able to run at 20 Hz, but the prediction quality degraded rapidly beyond roughly two to three seconds (Fig. 2.4) into the future, which made it unsuitable for the heave-landing use case.

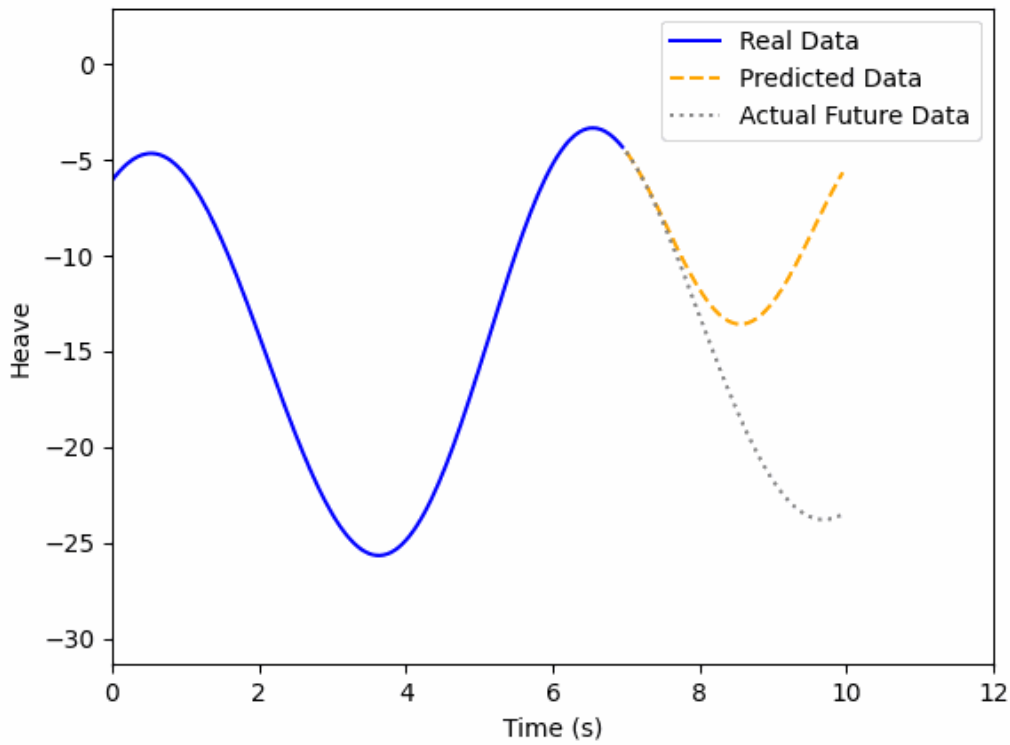


FIGURE 2.4: Snapshot of ARIMA-based predictions

2.3.2.2 Temporal Convolutional Network

A temporal convolutional network (TCN) [37] was then tested as a stronger sequence model. The tested configuration used a 40 s input window, corresponding to 800 samples at 20 Hz, to predict 8 s into the future, corresponding to 160 output samples, with two channels defining the temporal convolution stack. The TCN was

noticeably better than the ARIMA baseline but remained computationally heavy, and the models that were fast enough for the real-time loop did not achieve the desired short-horizon accuracy (Fig. 2.5).

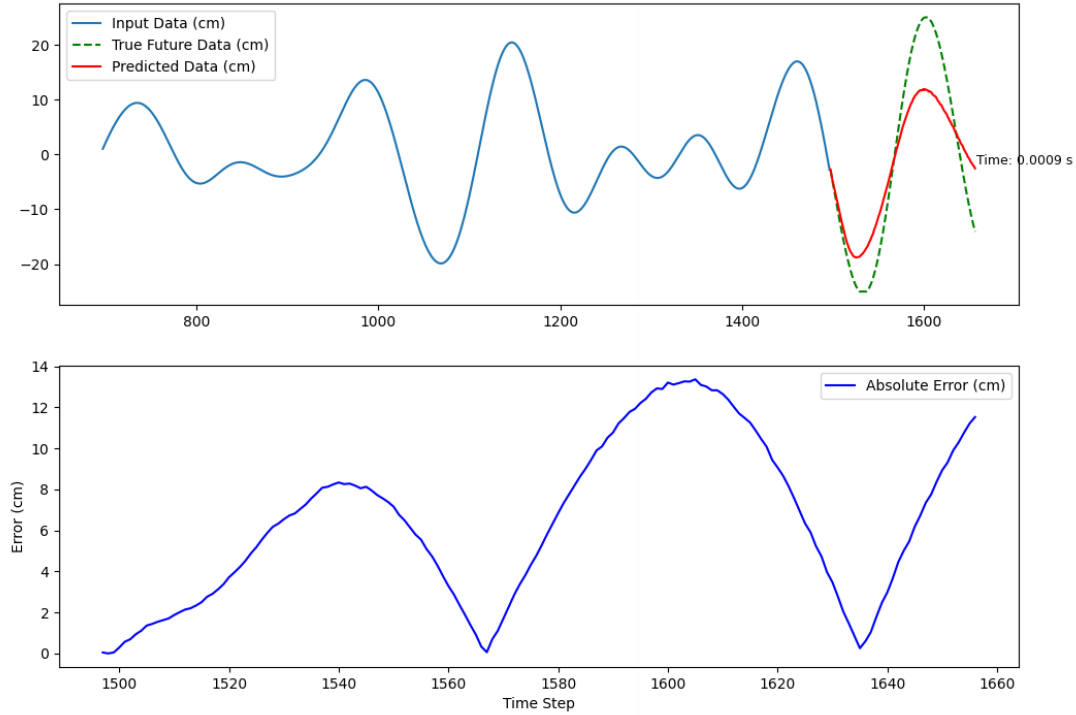


FIGURE 2.5: Snapshot of TCN-based predictions

2.3.2.3 Minor Component Analysis

Alongside autoregressive and convolutional baselines, a purely statistical predictor based on minor component analysis (MCA) was also implemented and tested. The starting point for this method was the ship-motion prediction work of Zhao *et al.* [38], where MCA was proposed as a linear long-horizon predictor for ship motion by exploiting the low-energy subspace of a windowed autocorrelation matrix. The method is closely related to principal component analysis, but instead of using the

dominant eigenvectors of the correlation structure, it uses the eigenvectors associated with the smallest eigenvalues.

Let each training window be written as

$$X_i = \begin{bmatrix} X_{1i} \\ X_{2i} \end{bmatrix} \in \mathbb{R}^{n+m}, \quad (2.2)$$

where $X_{1i} \in \mathbb{R}^n$ is the past heave history and $X_{2i} \in \mathbb{R}^m$ is the future segment to be predicted. Following [38], the autocorrelation matrix is formed as

$$R = \sum_{i=1}^{N_w} X_i X_i^T, \quad (2.3)$$

where N_w is the number of windowed training samples. If the P eigenvectors corresponding to the smallest eigenvalues are stacked into a matrix

$$B = [B_1 \ B_2], \quad (2.4)$$

then the MCA approximation enforces

$$B_1 X_{1i} + B_2 X_{2i} \approx 0, \quad (2.5)$$

which yields the linear future predictor

$$\hat{X}_{2i} \approx - (B_2^T B_2 + \lambda I)^{-1} B_2^T B_1 X_{1i}. \quad (2.6)$$

The internal MCA implementation uses a ridge-regularised version of this expression, together with row-wise centring of the window matrix before eigenanalysis for numerical stability.

The retained MCA model used a past window of $n = 800$ samples (40 s at 20 Hz), a future horizon of $m = 120$ samples (6 s at 20 Hz), unit stride, row-wise centring, an energy cutoff of 0.01, and ridge regularisation $\lambda = 10^{-6}$. On the current dataset, this energy cutoff retained $P = 906$ minor components. The training data consisted of four normalised heave records, while the fifth was held out for evaluation. Because MCA is a closed-form linear method, fitting the model required only a few seconds.

The MCA result is important for three reasons:

- on smooth motion-capture data, the short-horizon accuracy is strong, with 2.899 cm average absolute error over the first 3 s and 3.174 cm at 4 s, which is at par with GRU's performance (on smooth test data) ;
- the total 6 s error remains moderate at 4.343 cm while the average prediction time is effectively negligible, showing why linear statistical predictors remain attractive as a real-time baseline; and
- once realistic noise is injected, the predictor degrades substantially because the model is globally linear and cannot adapt its representation online (Fig. 2.6).

The failure of these linear and statistical models to achieve the desired accuracy, and TCN's inability to run on a Jetson Nano at 20 Hz, motivated the exploration of Recurrent Neural Networks, which are lightweight and retain system memory. LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) were explored and are discussed in the next section.

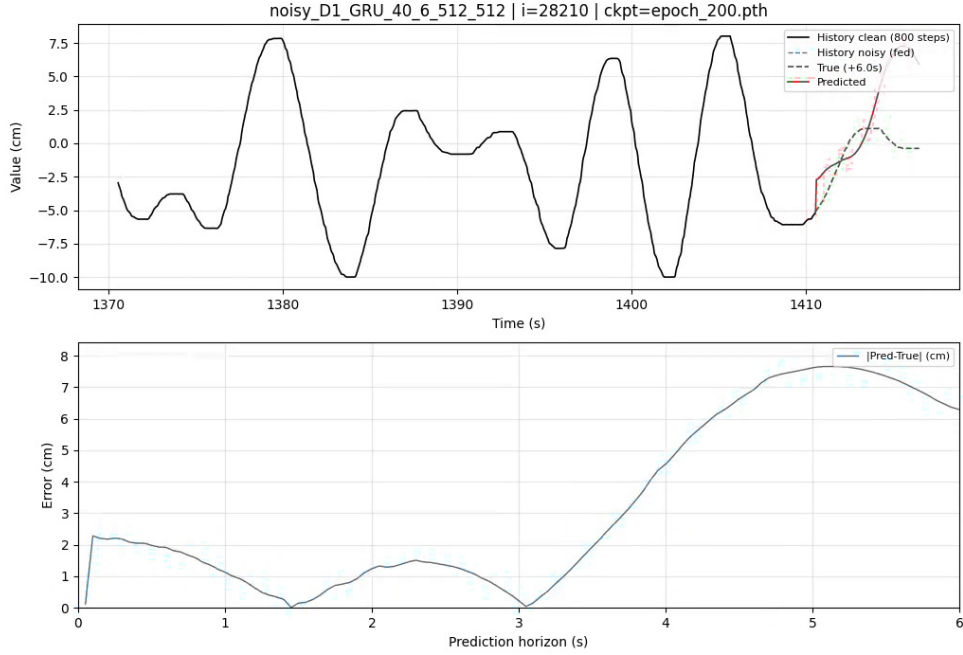


FIGURE 2.6: Snapshot of MCA-based predictions

2.3.3 Recurrent Predictors for Heave Forecasting

The main predictor family pursued in this thesis is recurrent, because the heave signal is sequential, partially periodic, and sensitive to phase error over the first few seconds of the forecast horizon.

2.3.3.1 Shared Training Objective for LSTM and GRU

Both recurrent model families were trained using the same supervised objective and the same early-horizon weighting logic. Each training sample contains an observed heave history of length N_{in} and a future target sequence of length N_{out} . Let

$$\mathbf{y} = [y_1, \dots, y_{N_{\text{out}}}]^T, \quad \hat{\mathbf{y}} = [\hat{y}_1, \dots, \hat{y}_{N_{\text{out}}}]^T \quad (2.7)$$

denote the true and predicted future heave signals, and let K denote the number of forecast steps corresponding to the first x seconds of the horizon:

$$K = \min(20x, N_{\text{out}}). \quad (2.8)$$

The implemented positional prediction loss is split into an early weighted segment and an unweighted remainder:

$$\mathcal{L}_{\text{pos}} = w \text{MSE}(\hat{\mathbf{y}}_{1:K}, \mathbf{y}_{1:K}) + \text{MSE}(\hat{\mathbf{y}}_{K+1:N_{\text{out}}}, \mathbf{y}_{K+1:N_{\text{out}}}), \quad (2.9)$$

with the understanding that the second term is omitted when $K = N_{\text{out}}$. This is exactly how the code computes the loss: it does *not* apply a single weighted mean over the whole horizon, but instead computes two separate MSE terms and multiplies only the first block by the weight w . The first x seconds are penalised more strongly because they are the most important part of the horizon for touchdown timing.

The recurrent models are also trained with continuity constraints tied to the observed tail of the input history. Let the last L observed samples be

$$\mathbf{o} = [o_{-L+1}, \dots, o_0]^T. \quad (2.10)$$

The first continuity term matches the initial forecast jump:

$$\mathcal{L}_{\text{jump}} = [(\hat{y}_1 - o_0) - (y_1 - o_0)]^2. \quad (2.11)$$

This expression is the logic behind the jump loss shown in the earlier sketch. Algebraically, the common o_0 cancels and the term reduces to $(\hat{y}_1 - y_1)^2$. However, it is written in jump form because the implementation explicitly enforces continuity at the boundary between the observed history and the predicted future. The loss is

therefore interpreted as “match the first predicted jump away from the last observed sample,” not merely “match the first future sample.”

The remaining continuity terms are also formed from this observation-to-forecast boundary. The code first concatenates the clean observed tail with the prediction and the target,

$$\hat{\mathbf{s}} = [o_{-L+1}, \dots, o_0, \hat{y}_1, \dots, \hat{y}_{N_{\text{out}}}]^T, \quad \mathbf{s} = [o_{-L+1}, \dots, o_0, y_1, \dots, y_{N_{\text{out}}}]^T, \quad (2.12)$$

and then computes first and second forward differences on these concatenated sequences. Define

$$\Delta \hat{s}_j = \hat{s}_{j+1} - \hat{s}_j, \quad \Delta s_j = s_{j+1} - s_j, \quad (2.13)$$

and the future-aligned slices

$$\Delta \hat{s}_j^f = \Delta \hat{s}_{L-1+j}, \quad \Delta s_j^f = \Delta s_{L-1+j}, \quad j = 1, \dots, N_{\text{out}}. \quad (2.14)$$

Using the same horizon weighting vector

$$\alpha_j = \begin{cases} w, & 1 \leq j \leq K, \\ 1, & K < j \leq N_{\text{out}}, \end{cases} \quad (2.15)$$

The velocity loss is

$$\mathcal{L}_{\text{vel}} = \frac{1}{N_{\text{out}}} \sum_{j=1}^{N_{\text{out}}} \alpha_j \left(\Delta \hat{s}_j^f - \Delta s_j^f \right)^2. \quad (2.16)$$

The acceleration term is defined analogously through second forward differences on the same concatenated sequences,

$$\Delta^2 \hat{s}_j = \Delta \hat{s}_{j+1} - \Delta \hat{s}_j, \quad \Delta^2 s_j = \Delta s_{j+1} - \Delta s_j, \quad (2.17)$$

with future-aligned slices

$$\Delta^2 \hat{s}_j^f = \Delta^2 \hat{s}_{L-2+j}, \quad \Delta^2 s_j^f = \Delta^2 s_{L-2+j}, \quad j = 1, \dots, N_{\text{out}}, \quad (2.18)$$

which yields

$$\mathcal{L}_{\text{acc}} = \frac{1}{N_{\text{out}}} \sum_{j=1}^{N_{\text{out}}} \alpha_j \left(\Delta^2 \hat{s}_j^f - \Delta^2 s_j^f \right)^2. \quad (2.19)$$

The continuity terms are introduced gradually over the first E_{warm} epochs using

$$\beta_e = \min \left(1, \frac{e}{E_{\text{warm}}} \right), \quad (2.20)$$

and the total training objective is

$$\mathcal{L} = \mathcal{L}_{\text{pos}} + \beta_e (\lambda_{\text{jump}} \mathcal{L}_{\text{jump}} + \lambda_{\text{vel}} \mathcal{L}_{\text{vel}} + \lambda_{\text{acc}} \mathcal{L}_{\text{acc}}). \quad (2.21)$$

In the retained training configuration, the default continuity weights are

$$\lambda_{\text{jump}} = 8.0, \quad \lambda_{\text{vel}} = 3.0, \quad \lambda_{\text{acc}} = 0.8, \quad (2.22)$$

with a tail observation length of $L = 5$ samples and continuity warmup over $E_{\text{warm}} = 80$ epochs. The same objective is used for both the LSTM and GRU sweeps, so the two recurrent families differ primarily in architecture and parameterisation rather than in the loss function itself.

2.3.3.2 LSTM Predictor Family

Long short-term memory (LSTM) networks [39] were evaluated because they are a standard recurrent architecture for long-horizon sequence prediction. An LSTM augments the hidden state with a separate cell state and three gates: an input gate

that determines which new information should be written into the cell, a forget gate that determines which old information should be discarded, and an output gate that determines how much of the cell state should be exposed as the hidden representation. This gating structure makes LSTMs more robust than plain recurrent networks on long sequences and also solves the vanishing gradient issue in RNNs, but it also increases the parameter count and runtime relative to lighter gated architectures such as GRUs.

The current LSTM sweep used the same overall sequence-prediction framing as the GRU sweep: a heave-history input window of 800 samples, a 120-sample future horizon, and loss weighting that emphasises the early part of the prediction window. When multiple completed trainings existed for the same architecture, the best-performing one was retained. The best retained LSTM results are summarised in Table 2.1.

The strongest LSTM configuration in the current sweep was the four-layer model with hidden sizes [512, 512, 512, 512], which achieved 2.99 cm average absolute error over the first 3 s, 3.24 cm at 4 s, 3.79 cm total error, and 0.704 ms average prediction time. A two-layer [1024, 1024] LSTM followed closely with 3.89 cm total error and a lower average prediction time of 0.291 ms. These results confirm that LSTMs remain competitive for the heave-prediction problem, but at a larger parameter burden than the GRU models ultimately retained for the thesis pipeline. An **important setback** of the prediction-time approach throughout the report is that it was tested on either an A6000 or an RTX5090, rather than on a Jetson Nano. That is another reason why multiple three- and four-layer networks performed very well in terms of accuracy but later could not be deployed on a Jetson Nano.

TABLE 2.1: Best LSTM configurations from the current heave-prediction log sweep. Each row reports the best completed run retained for that architecture.

Hidden Sizes	Trainable Parameters	Avg. 3 s Error (cm)	Avg. 4 s Error (cm)	Total Abs. Error (cm)	Pred. Time (ms)
[512, 512, 512, 512]	7,420,024	2.989	3.244	3.789	0.704
[1024, 1024]	12,726,392	3.351	3.422	3.892	0.291
[2048, 1024]	29,515,896	3.196	3.400	3.961	0.301
[1024, 1024, 1024]	21,123,192	3.316	3.456	3.966	0.674
[1024, 512, 256]	8,175,736	3.435	3.542	4.016	1.477
[1024, 512, 512]	9,519,224	3.285	3.488	4.032	0.668
[512, 512, 512]	5,318,776	3.393	3.513	4.071	0.643
[1024, 512]	7,417,976	4.636	4.426	4.507	0.370

2.3.3.3 GRU Predictor Family

The final predictor family retained for the heave-landing pipeline was a multi-layer gated recurrent unit (GRU) [40] trained on normalised heave histories. A GRU can be viewed as a simplified gated recurrent alternative to the LSTM: instead of maintaining a separate cell state with input, forget, and output gates, it uses a hidden state together with update and reset gates to control how much past information is preserved and how much new information is incorporated. In practice, this often yields accuracy close to that of an LSTM while reducing parameter count and inference overhead, which is attractive for a real-time landing loop.

For an input x_t and previous hidden state h_{t-1} , the GRU update at time step t is

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad (2.23)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \quad (2.24)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \quad (2.25)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad (2.26)$$

where z_t is the update gate, r_t is the reset gate, and $\odot$ denotes elementwise multiplication. The predictor used in this thesis stacks multiple GRU layers, applies dropout between layers, and uses a linear output head followed by a tanh activation

so that the predicted future sequence remains inside the normalised range used for training.

The heave data were scaled to the interval $[-1, 1]$ before training, since the network predicted more stably in that range. In the original development, this normalised output was mapped back to physical motion by multiplying by the platform scale, which was taken as ± 25 cm for the target emulator motion envelope.

More than 40 GRU models were trained, most for 300–400 epochs and some for substantially longer runs, on two NVIDIA RTX A6000 GPUs. The hyperparameter sweep varied:

- input sequence length,
- output sequence length,
- GRU hidden-layer depth and width,
- learning rate, and
- the weight applied to the first few seconds of the prediction loss.

Two GRU operating modes were investigated. In the first, the model receives the full input window at every control step and directly predicts the future horizon. In the second, a sequential GRU receives one new point per time step while propagating its hidden state forward. The direct-input formulation is computationally more expensive because the entire history window is reprocessed each time, but it provides better short-horizon accuracy, i.e., one can start using the predictions immediately. The sequential formulation eventually produced useful forecasts once enough history had accumulated, but it was weaker at the beginning of each sequence, which is undesirable for a controller that must act immediately when deck motion changes.

Finally, due to the low computational demand, the latter was chosen and the UAV was made to hover for some time before the predictions could be trusted.

The recurrent input is an 800-sample history, and the output is a 120-sample forecast (running at 20 Hz). The implementation supports both a univariate input containing deck height alone and a two-channel variant containing height and estimated vertical velocity. The training loop uses Adam with an exponentially decaying learning rate, a decay factor of 0.998 per epoch, and gradient clipping safeguards. At each epoch, the model iterates over batches of sequence data, computes the shared loss described above, performs backpropagation, and stores periodic checkpoints and evaluation summaries.

2.3.3.4 Results from the GRU Sweep

Table 2.2 summarises the GRU sweep. The table reports the best-performing checkpoint for each shortlisted hidden-size configuration, ranked by total average absolute error.

Two results are especially important. The best total-error run in the current sweep is the two-layer model with hidden sizes [1024, 1024], which achieved 2.95 cm average absolute error over the first 3 s, 3.13 cm at 4 s, and 3.74 cm total error, with an average prediction time of 0.305 ms. This slightly outperforms the best LSTM configuration in Table 2.1 while also using fewer parameters than the strongest two-layer LSTM competitor. A smaller three-layer model with hidden sizes [512, 256, 128] achieved 3.66 cm at 3 s, 3.96 cm at 4 s, and 4.54 cm total error with only 1.55 M trainable parameters and 0.661 ms average prediction time.

TABLE 2.2: Best GRU configurations from the current heave-prediction log sweep.
Each row reports the best completed run retained for that architecture.

Hidden Sizes	Trainable Parameters	Avg. 3 s Error (cm)	Avg. 4 s Error (cm)	Total Abs. Error (cm)	Pred. Time (ms)
[1024, 1024]	9,575,544	2.952	3.130	3.742	0.305
[2048, 1024]	22,167,672	3.383	3.556	4.017	0.605
[512, 512, 512, 512]	5,580,408	3.669	3.886	4.389	0.708
[512, 512, 512]	4,004,472	3.682	3.895	4.407	0.638
[512, 256, 128]	1,546,104	3.665	3.960	4.540	0.661
[1024, 512, 256]	6,139,512	4.176	4.354	4.734	0.663
[512, 512, 512, 128]	4,204,920	4.121	4.321	4.769	0.716
[1024, 512, 256, 128]	6,272,376	3.983	4.234	4.834	0.828

The qualitative behaviour of representative GRU models is shown in Fig. 2.7. The figure illustrates the progressive improvement obtained by increasing the representational capacity of the recurrent stack. Larger models improve long-horizon stability, but the comparison in Table 2.2 shows that simply scaling the hidden state is not sufficient on its own; architecture, weighting of the early prediction window, and training configuration all matter.

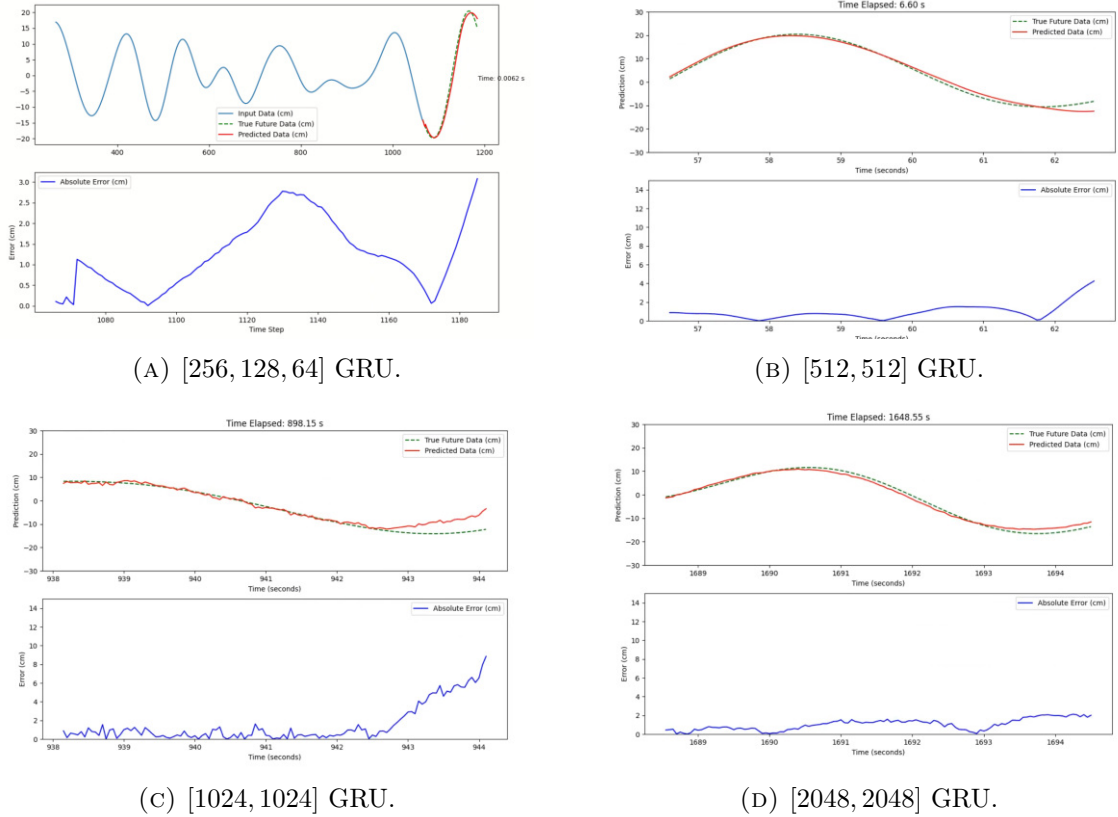


FIGURE 2.7: Qualitative comparison of GRU predictors from the heave-model sweep.

2.3.4 Final Predictor and Peak-Target Selection

2.3.4.1 Final Motion Predictor model

The final predictor retained for the landing pipeline and all the hardware experiments was the two-layer GRU with hidden sizes $[1024, 1024]$. This model produced the best overall combination of short-horizon accuracy, total forecast accuracy, and sub-millisecond inference time. A final sanity check was done by running the model on board the Jetson Nano, which easily ran at 20 Hz.

A non-intuitive result of the recurrent model sweep was that increasing the trainable parameters did not improve accuracy, and the tuning process was brute-force trial and error. This research does not claim that the current chosen model is optimal. A very detailed study of sea motion prediction is available, including methods such as physics-based models, CFD-based models, and dedicated ocean models that could have been used to improve the accuracy and robustness of the predictions. These methods, however, are outside the scope of this study.

2.3.4.2 Peak-Target Selection

The goal of the prediction was to find a timestamp in future, where the motion of the platform is flat, which is possible when either the platform is at its highest point or lowest. The highest point was chosen for landing to reduce the risk of collision. At each control update, the predictor receives the recent deck-heave history and produces a future forecast

$$\hat{\mathbf{z}}_{\text{fut}} = [\hat{z}_d(t + \Delta t), \hat{z}_d(t + 2\Delta t), \dots, \hat{z}_d(t + T_p)]. \quad (2.27)$$

Rather than committing to touchdown at an arbitrary future instant, the landing planner searches this predicted heave trajectory for a favourable future local maximum,

$$t^* = \arg \max_{t \in \mathcal{T}_{\text{cand}}} \hat{z}_d(t), \quad (2.28)$$

where $\mathcal{T}_{\text{cand}}$ is the admissible future time window inside the optimiser horizon. The intuition is straightforward: near a deck-heave maximum, the platform's vertical velocity is close to zero, so the relative touchdown speed can be reduced even when the vehicle continues descending.

The implementation used in the landing controller updates the predicted trajectory at 20 Hz and stores it in a fixed-length future buffer. The deployed peak-detection logic first applies a 7-point moving average to the future heave forecast,

$$\bar{z}_j = \frac{1}{7} \sum_{\ell=0}^6 \hat{z}_{d,j-\ell}, \quad (2.29)$$

using the valid interior of the predicted sequence. It then inspects the last 10 smoothed samples and selects the candidate (if any). A peak is accepted only if that candidate is the maximum of the suffix window and if it lies outside a 0.5 s cooldown period from the previous accepted peak. Once a valid peak has been identified, my Landing Trajectory controller is initiated only after that peak enters the prediction horizon of the optimiser. Algorithm 1 summarises the exact peak-selection rule used by the deployed landing generator.

2.4 Vertical Trajectory Generator

The vertical landing controller is implemented as a short-horizon receding-horizon optimisation running at 20 Hz. The landing pipeline uses an acados-based [41]

Algorithm 1 Prediction-Driven Peak Selection for Heave Landing

Require: Predicted heave buffer $\hat{\mathbf{z}}_{\text{fut}}$, current time t , sampling time Δt , minimum prominence δ_{prom} , cooldown T_{cd} , last accepted-peak time t_{last}

Ensure: Candidate touchdown peak $(z_{\text{peak}}, t_{\text{peak}})$ or rejection

```

1: if  $\text{len}(\hat{\mathbf{z}}_{\text{fut}}) < 7$  then
2:   return no peak
3: end if
4: Compute 7-point valid moving average  $\bar{\mathbf{z}} = \text{MA}_7(\hat{\mathbf{z}}_{\text{fut}})$ 
5: if  $\text{len}(\bar{\mathbf{z}}) < 10$  then
6:   return no peak
7: end if
8: Extract the last 10 smoothed samples:  $\bar{\mathbf{z}}_{\text{tail}} \leftarrow \bar{\mathbf{z}}[-10 : ]$ 
9: Use the fixed candidate index  $j_c \leftarrow 4$  inside  $\bar{\mathbf{z}}_{\text{tail}}$ 
10: if  $\bar{z}_{\text{tail}, j_c} \neq \max(\bar{\mathbf{z}}_{\text{tail}})$  then
11:   return no peak
12: end if
13: Recover indices  $j_s \leftarrow \text{len}(\bar{\mathbf{z}}) - 10 + j_c$  and  $j_p \leftarrow j_s + 3$ 
14: Accept the peak with  $z_{\text{peak}} \leftarrow \bar{z}_{j_s}$  and  $t_{\text{peak}} \leftarrow t + j_p \Delta t$ 
15: return  $(z_{\text{peak}}, t_{\text{peak}})$ 

```

vertical landing generator, where the optimisation is built as a discrete-time problem with state $x_k = [z_k, v_k]^T$ and control input u_k representing commanded vertical acceleration. The horizon length is $H = 90$ steps with sampling time $\Delta t = 0.05$ s, giving a 4.5 s prediction horizon.

2.4.1 Discrete Vertical Dynamics and Reference Construction

The vertical motion model used by the optimiser is the double-integrator system

$$z_{k+1} = z_k + \Delta t v_k, \quad (2.30)$$

$$v_{k+1} = v_k + \Delta t u_k, \quad (2.31)$$

where z_k , v_k , and u_k denote the UAV vertical position, velocity, and commanded acceleration at step k . The initial state is fixed to the measured current state,

$$z_0 = z_{\text{curr}}, \quad v_0 = v_{\text{curr}}. \quad (2.32)$$

Once a future heave maximum has been selected, the controller forms a reference altitude trajectory by linearly interpolating from the current altitude z_{curr} to the selected peak altitude z_{peak} over the available number of steps before the peak occurs, and then holding the reference constant at the peak level for the remainder of the horizon. This produces a descent profile that is simple to optimise and still explicitly aligned with the predicted touchdown opportunity.

2.4.2 Receding-Horizon Landing Objective

The vertical optimiser is implemented exactly as a stage-wise finite-horizon problem in acados. For the running horizon of length H with $N = H - 1$ control stages, the objective is

$$\begin{aligned} \min_{\{x_k, u_k\}} \quad J = & \sum_{k=0}^{N-1} \left[w_{\text{track}}(z_k - z_k^{\text{ref}})^2 + (w_v + w_{\text{tail},k})v_k^2 + w_u u_k^2 + w_{\text{fp},k}(z_k - z_{\text{peak}})^2 \right] \\ & + \left[w_{\text{track}}(z_N - z_N^{\text{ref}})^2 + (w_v + w_{\text{tail},N})v_N^2 + w_{\text{fp},N}(z_N - z_{\text{peak}})^2 \right] \end{aligned} \quad (2.33)$$

subject to

$$\begin{aligned}
x_0 &= \begin{bmatrix} z_{\text{curr}} & v_{\text{curr}} \end{bmatrix}^T, \\
z_{k+1} &= z_k + \Delta t v_k, & k = 0, \dots, N-1, \\
v_{k+1} &= v_k + \Delta t u_k, & k = 0, \dots, N-1, \\
v_{\min} &\leq v_k \leq v_{\max}, & k = 0, \dots, N, \\
u_{\min} &\leq u_k \leq u_{\max}, & k = 0, \dots, N-1.
\end{aligned} \tag{2.34}$$

The first activated term, $w_{\text{fp},k}$, penalises position error relative to the selected peak altitude. The second, $w_{\text{tail},k}$, increases the penalty on late-horizon velocity so that the optimiser suppresses residual descent speed near touchdown.

The peak-alignment index is chosen from the predicted time-to-peak as

$$k_f = \min\left(H-1, \left\lceil \frac{t_{\text{peak}} - t_{\text{now}}}{\Delta t} \right\rceil - 1\right), \tag{2.35}$$

and the tail-penalty window begins at

$$k_t = \max(0, k_f - N_{\text{tail}} + 1). \tag{2.36}$$

The linearly interpolated reference trajectory is therefore

$$z_k^{\text{ref}} = \begin{cases} z_{\text{curr}} + \frac{k}{k_f} (z_{\text{peak}} - z_{\text{curr}}), & 0 \leq k \leq k_f, \\ z_{\text{peak}}, & k_f < k \leq H-1, \end{cases} \tag{2.37}$$

for $k_f \geq 1$, with the understanding that the implementation uses a numerically constructed `linspace` version of the same profile and falls back to an immediate peak-level reference when the predicted peak is already at the current control instant.

The state and input limits used in the controller are

$$v_{\min} = -0.6 \text{ m/s}, \quad v_{\max} = 1.5 \text{ m/s}, \quad (2.38)$$

$$u_{\min} = -2 \text{ m/s}^2, \quad u_{\max} = 2 \text{ m/s}^2. \quad (2.39)$$

The optimisation is transcribed in acados as a discrete nonlinear program with SQP iterations, exact Hessian approximation, and partial condensing through HPIPM. The stage parameter vector used in the solver is

$$p_k = [z_k^{\text{ref}}, z_{\text{peak}}, w_{\text{final,pos},k}, w_{\text{tail},k}, \delta_{\text{cbf},k}, z_{p,0}, v_{p,0}, a_{p,0}]^T, \quad (2.40)$$

where $w_{\text{final,pos},k}$ is activated only at the peak-alignment stage, $w_{\text{tail},k}$ is activated only over the terminal braking window, and $\delta_{\text{cbf},k}$ enables the barrier term when safety filtering is active. At each control tick, the current state is hard-constrained as the stage-0 state, the horizon reference is refreshed using the latest predicted peak, and the first optimised velocity command is published to the vertical controller interface.

The optimiser parameters used in the present landing pipeline are summarised in Table 2.3.

The safety constraint associated with relative stopping distance is not treated as part of the nominal objective above. Instead, it is enforced as a dedicated barrier-based constraint in the next section, where it acts as a safety filter on the otherwise prediction-driven descent plan.

TABLE 2.3: Vertical MPC configuration used in the heave-landing controller.

Parameter	Value	Role
Δt	0.05 s	Control/update period (20 Hz)
H	90	Prediction horizon length
z_{hover}	3.2 m	Initial hover altitude before descent
$v_{\min}, v_{\max}$	$[-0.6, 1.5]$ m/s	Vertical velocity bounds
$u_{\min}, u_{\max}$	$[-2, 2]$ m/s ²	Vertical acceleration bounds
w_{track}	10.0	Reference-tracking penalty
w_u	0.1	Acceleration effort penalty
w_v	1.0	Global vertical-velocity penalty
$w_{\text{final,pos}}$	50.0	Peak-alignment position penalty
w_{tail} (sim)	600.0	Terminal velocity penalty in simulator mode
w_{tail} (experiment)	2400.0	Terminal velocity penalty in ground experiments
N_{tail} (sim)	22	Terminal braking window length in simulator mode
N_{tail} (experiment)	40	Terminal braking window length in ground experiments

2.5 Safe Landing Logic and Safety Filter

For real-world deployment, autonomous landing algorithms must include explicit safety guarantees, particularly because such operations may occur near personnel or involve the transportation of high-value payloads. In the present framework, the landing decision depends on GRU-based prediction of the platform heave motion, especially for estimating suitable touchdown instants near motion peaks. However, due to system nonlinearities, modelling errors, sensor noise, and sudden external disturbances, the GRU prediction may either overestimate or underestimate the actual platform motion.

An overprediction is generally less critical: the UAV may remain above the platform, and the landing attempt may fail, but the system is unlikely to suffer physical damage. In contrast, an underprediction is safety-critical because the actual platform may rise higher than expected, reducing the relative separation and potentially causing the UAV to collide with the deck. Therefore, the landing controller requires a fast safety-monitoring mechanism that can detect such unsafe conditions online

and abort or modify the descent before collision becomes unavoidable. This motivates the incorporation of a Control Barrier Function (CBF)-based safety filter in the landing framework.

2.5.1 Control Barrier Function for Vertical Landing Safety

To prevent the UAV from committing to a descent that cannot be safely arrested before touchdown, the vertical landing controller is augmented with a control barrier function (CBF). The general safety setting follows the standard control-affine model

$$\dot{x} = f(x, t) + g(x, t)u, \quad (2.41)$$

with safe set

$$\mathcal{C} = \{x \in \mathbb{R}^n \mid h(x) \geq 0\}. \quad (2.42)$$

For a continuously differentiable barrier function h , a CBF satisfies

$$\sup_{u \in \mathcal{U}} [L_f h(x, t) + L_g h(x, t) u + \alpha(h(x))] \geq 0, \quad (2.43)$$

and we choose the standard linear class- $\mathcal{K}$ function

$$\alpha(h) = \gamma h, \quad \gamma > 0, \quad (2.44)$$

so the enforced safety condition becomes

$$\dot{h}(x, u) + \gamma h(x) \geq 0. \quad (2.45)$$

This is the barrier formulation used throughout the implemented landing generator, following the general theory of Ames *et al.* [42, 43].

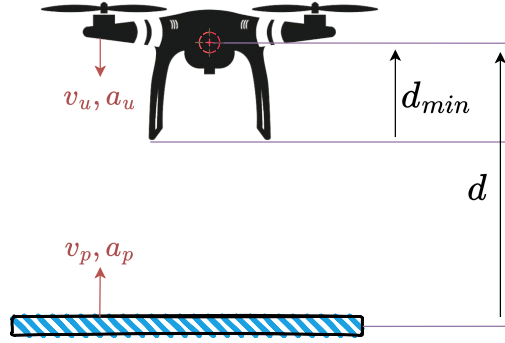


FIGURE 2.8: Visualisation of CBF parameters

For the heave-landing problem, let z_u denote the vertical position of the UAV centre of gravity and let z_p denote the platform-surface height at the intended touchdown point. Define the relative vertical distance

$$d := z_u - z_p, \quad (2.46)$$

and let $d_{\min} > 0$ denote the fixed distance from the UAV centre of gravity to the lowest point of the landing gear. The available stopping clearance is then

$$c := d - d_{\min}. \quad (2.47)$$

The setup is described in Figure 2.8. Define the relative vertical velocity by

$$v_r := \dot{d} = v_u - v_p, \quad (2.48)$$

where v_u and v_p are the UAV and platform vertical velocities. During descent, the potentially dangerous regime is $v_r < 0$, meaning that the vehicle is closing on the

deck. The relative vertical dynamics are

$$\dot{d} = v_r, \quad (2.49)$$

$$\dot{v}_r = u - a_p, \quad (2.50)$$

where u is the commanded UAV vertical acceleration and a_p is the platform vertical acceleration.

Let $v_L \geq 0$ denote the maximum admissible touchdown relative-speed magnitude and let $a_{\text{rel,max}} > 0$ denote a conservative bound on the available relative braking acceleration. The stopping distance required to reduce the closing speed from v_c to v_L is

$$d_{\text{stop}}(v_r) := \max\left(0, \frac{v_c^2 - v_L^2}{2a_{\text{rel,max}}}\right), \quad (2.51)$$

with $v_c = \max(0, -v_r)$. **The landing state is safe only when the available clearance exceeds the required stopping distance:**

$$(d - d_{\min}) - d_{\text{stop}}(v_r) \geq 0. \quad (2.52)$$

This motivates the safe set

$$\mathcal{C} = \{(d, v_r) \mid (d - d_{\min}) - d_{\text{stop}}(v_r) \geq 0\}, \quad (2.53)$$

and the barrier function

$$h(d, v_r) := (d - d_{\min}) - d_{\text{stop}}(v_r). \quad (2.54)$$

During active descent, the controller uses a smooth approximation of the negative relative velocity,

$$v_r^- := \frac{1}{2} \left(v_r - \sqrt{v_r^2 + \varepsilon} \right), \quad \varepsilon > 0, \quad (2.55)$$

which approximates $\min(v_r, 0)$ and avoids nonsmooth optimization behavior. The implemented barrier terms are therefore

$$d_{\text{stop}} = \frac{(v_r^-)^2 - v_L^2}{2a_{\text{rel,max}}}, \quad (2.56)$$

$$h(d, v_r) = (d - d_{\text{min}}) - d_{\text{stop}}, \quad (2.57)$$

with the corresponding time derivative

$$\dot{h} = v_r^- - \frac{v_r^-}{a_{\text{rel,max}}}(u - a_p). \quad (2.58)$$

The safety filter then enforces

$$h \geq 0, \quad \dot{h} + \gamma h \geq 0, \quad (2.59)$$

which preserves forward invariance of the chosen safe set under the assumed relative vertical dynamics.

In the acados implementation, the barrier constraint is enforced only at stage 0 of the optimisation at each control tick. The optimizer is parameterized with the current platform height $z_{p,0}$, platform vertical velocity $v_{p,0}$, and platform vertical

acceleration $a_{p,0}$, and the stage-0 constraint is written as

$$\begin{aligned}
z_{p,0} &:= z_{\text{marker}}, & d_0 &:= z_0 - z_{p,0}, & v_{r,0} &:= v_0 - v_{p,0}, \\
v_{r,0}^- &:= \frac{1}{2} \left(v_{r,0} - \sqrt{v_{r,0}^2 + \varepsilon} \right), \\
d_{\text{stop}} &:= \frac{(v_{r,0}^-)^2 - v_{\text{land}}^2}{2a_{\text{rel}}}, & h_0 &:= (d_0 - d_{\text{min}}) - d_{\text{stop}}, \\
\dot{h}_0 &:= v_{r,0}^- - \frac{v_{r,0}^-}{a_{\text{rel}}} (u_0 - a_{p,0}), \\
\dot{h}_0 + \gamma h_0 &\geq 0, & h_0 &> 0.
\end{aligned} \tag{2.60}$$

The controller parameters used in the current implementation are

$$a_{\text{rel,max}} = 0.7 \text{ m/s}^2, \quad \varepsilon = 10^{-4}, \tag{2.61}$$

which are tuned empirically. These are summarised in Table 2.4.

TABLE 2.4: Barrier-function and supervisory parameters used in the safe-landing filter.

Parameter	Value	Role
$a_{\text{rel,max}}$	0.7 m/s ²	Conservative relative braking capability
ε	10 ⁻⁴	Smoothing term for v_r^-
d_{min} (sim)	0.12 m	Hard clearance in simulator mode
d_{min} (experiment)	0.27 m	Hard clearance in ground experiments
γ (sim)	4.0	Barrier gain in simulator mode
γ (experiment)	2.24	Barrier gain in ground experiments
v_L (sim)	0.25 m/s	Maximum admissible landing speed in simulator mode
v_L (experiment)	0.20 m/s	Maximum admissible landing speed in ground experiments
t_{hover}	10.0 s	Initial hover dwell before peak search
t_{return}	4.0 s	Wait time after aborted landing attempt

2.5.2 Operational Landing State Machine and Fallback Logic

The landing controller is not just an optimiser. It is embedded in an explicit state machine that manages when descent is allowed, when the controller should wait,

and when it should abandon the current landing attempt and return to a safe hover.

The implemented state sequence is:

- **INITIAL_HOVER**: the vehicle drives to the desired hover altitude, nominally 3.2 m;
- **PEAK_SEARCH**: the controller holds altitude while searching the predicted heave trajectory for a valid future peak;
- **WAIT_PEAK_IN_RANGE**: a detected peak is held until it enters the optimiser horizon;
- **LAND_TRAJECTORY**: the receding-horizon optimiser commands the descent toward the selected touchdown instant;
- **GO_BACK**: if optimisation fails or the peak window expires, the vehicle returns to hover;
- **GO_BACK_WAIT**: the controller waits for a fixed settling time before re-entering peak search; and
- **DONE**: touchdown is declared once the altitude difference to the marker falls below the landing tolerance and the vehicle is disarmed.

Several numerical thresholds govern this logic. A peak is considered usable only if it lies inside the optimiser horizon $H\Delta t = 4.5$ s. If the selected peak passes by more than 1 s before touchdown is achieved, the descent is aborted, and the vehicle returns to the hover state. After a failed attempt, the controller remains in the go-back wait state for 4 s before searching again. Touchdown is declared when the altitude difference falls below a mode-dependent tolerance: 0.20 m in gazebo mode and 0.36 m in the ground-experiment configuration.

Algorithm 2 shows the supervisory logic exactly as used in the controller: peak detection is first used to choose a touchdown opportunity, that peak is held until it enters the 4.5 s optimization horizon, and the optimizer is then re-solved at every control tick until touchdown or abort.

2.6 Results and Discussion

2.6.1 Simulation Validation of the MPC and CBF

Before the main hardware experiments, the vertical planner and the barrier-based safety logic were validated in simulation using Gazebo, ArduPilot SITL, the Iris quadrotor, and a Stewart-platform-based moving deck. The simulated environment is shown in Fig. 2.9. These tests were used to verify two distinct properties of the landing pipeline: first, that the control barrier condition prevents unsafe descent commitment when the predicted touchdown opportunity is severely biased; and second, that the GRU-MPC-CBF chain can produce a successful landing when the platform state is provided without estimation error.

The first simulation case intentionally injects a large prediction mismatch. The optimiser is given a touchdown target whose peak is offset by approximately 1.0 m below the true platform peak. Under this mismatch, a descent policy without a safety check would continue toward an infeasible touchdown condition and eventually collide with the rising platform. Instead, the barrier condition detects that the remaining stopping margin has become unsafe and prevents the controller from committing to that descent. This behaviour is reflected in the MPC and CBF traces of Fig. 2.10, where the planner trajectory is rejected before unsafe contact occurs.

Algorithm 2 Peak-Guided Safe Heave-Landing Supervisor

Require: Hover altitude z_{hover} , optimizer horizon $H\Delta t$, landing tolerance $\varepsilon_{\text{land}}$, return-wait time T_{return}

Ensure: Safe vertical command sequence until landing or hover recovery

```

1: Initialize state  $\leftarrow$  INITIAL_HOVER
2: while state  $\neq$  DONE do
3:   if state = INITIAL_HOVER then
4:     Drive  $z_u$  to  $z_{\text{hover}}$  with saturated vertical command
5:     if  $|z_u - z_{\text{hover}}| < 0.1$  then
6:       state  $\leftarrow$  PEAK_SEARCH
7:     end if
8:   else if state = PEAK_SEARCH then
9:     Hold altitude near  $z_{\text{hover}}$ 
10:    if Algorithm 1 returns  $(z_{\text{peak}}, t_{\text{peak}})$  then
11:      state  $\leftarrow$  WAIT_PEAK_IN_RANGE
12:    end if
13:  else if state = WAIT_PEAK_IN_RANGE then
14:     $\tau_{\text{peak}} \leftarrow t_{\text{peak}} - t$ 
15:    if  $0 < \tau_{\text{peak}} \leq H\Delta t$  then
16:      state  $\leftarrow$  LAND_TRAJECTORY
17:    else if  $\tau_{\text{peak}} \leq 0$  then
18:      Reject expired peak and return to PEAK_SEARCH
19:    else
20:      Hold altitude near  $z_{\text{hover}}$ 
21:    end if
22:  else if state = LAND_TRAJECTORY then
23:    Solve the vertical MPC using  $(z_u, v_u, z_{\text{peak}}, \tau_{\text{peak}})$  with the stage-0 CBF
    constraint active
24:    if  $|z_u - z_d| < \varepsilon_{\text{land}}$  then
25:      Command zero vertical velocity, disarm, state  $\leftarrow$  DONE
26:    else if MPC solve fails then
27:      state  $\leftarrow$  GO_BACK
28:    else if  $\tau_{\text{peak}} \leq -1.0$  then
29:      Reject stale touchdown window and set state  $\leftarrow$  GO_BACK
30:    else
31:      Publish the first optimised vertical-velocity command
32:    end if
33:  else if state = GO_BACK then
34:    Command climb/hold until  $|z_u - z_{\text{hover}}| \leq 0.05$ 
35:    if  $|z_u - z_{\text{hover}}| \leq 0.05$  then
36:      state  $\leftarrow$  GO_BACK_WAIT,  $t_0 \leftarrow t$ 
37:    end if
38:  else if state = GO_BACK_WAIT then
39:    Hold altitude near  $z_{\text{hover}}$ 
40:    if  $t - t_0 \geq T_{\text{return}}$  then
41:      Clear stored peak variables and return to PEAK_SEARCH
42:    end if
43:  end if
44: end while

```

The second simulation case uses the true platform position as the state input to the landing controller while the future heave motion is predicted by the GRU model. In this setting, the combined predictor, optimiser, and safety filter complete the landing successfully, with a final touchdown velocity of approximately -0.22 m/s. Together, these two simulations validate that the vertical MPC can guide the descent toward a predicted touchdown opportunity, while the CBF layer remains responsible for rejecting infeasible descent windows when prediction error becomes too large.

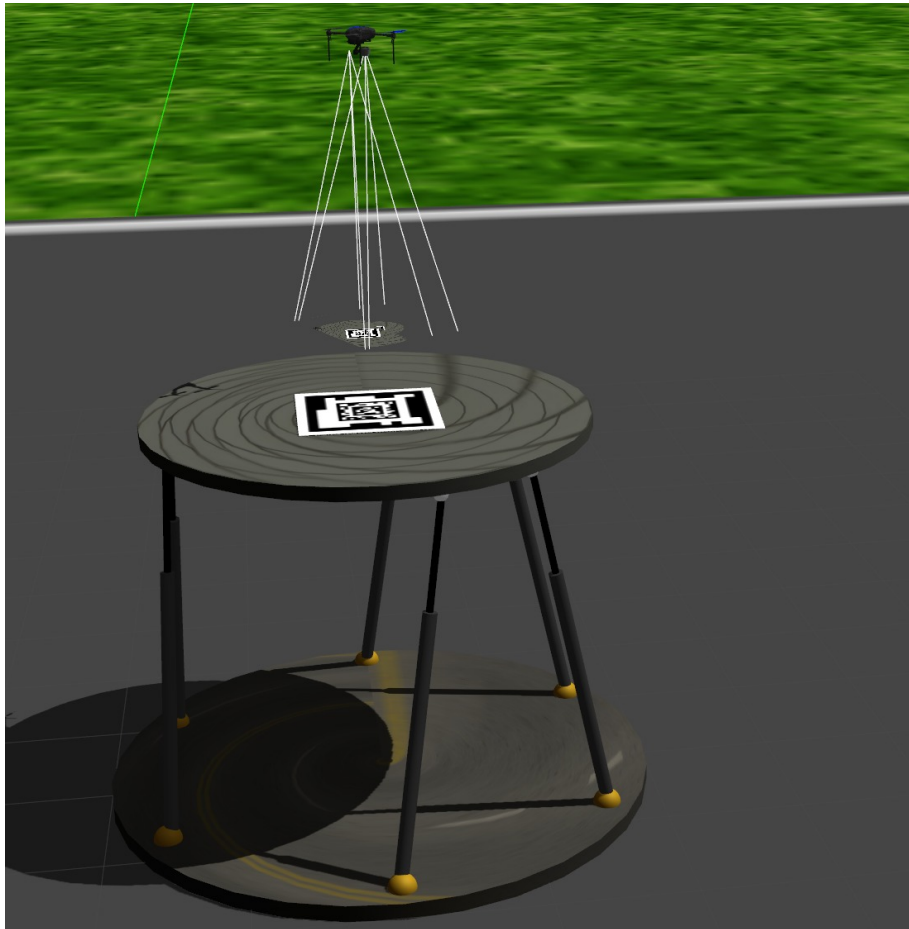


FIGURE 2.9: Gazebo simulation environment with the Iris UAV and the Stewart-platform-based moving deck.

2.6.2 Hardware Demonstration of the landing pipeline

As mentioned, the main experiments are part of the PhD thesis of Mr Chiranjeev Prachand, who developed the vision-based position estimation system. The contribution of this work is the use of the platform estimation data to generate a safe descent logic for heave landing. The Ship Deck emulator, along with safety nets, was placed for the experiments (Fig. 2.11). The landing pipeline consists of predicting the platform's motion using a GRU, then landing on the peak found by Algorithm 1 using the Vertical Trajectory Generator (Algorithm 2).

A total of 11 experiments were done for validation, all resulting in successful landing. Of the 11 successful vision-based tests, Cases 4 and 11 are included in this report for completeness as they showcase the importance of the CBF-based safety filter. Each case is reported with the same four plots:

- (A) compares the vision data with true MoCap data for the full time history of the landing
- (B) compares world frame altitudes for the full time history of the landing
- (C) plots world frame position and velocity performance as solved by the MPC from the time when the MPC node is started
- (D) plots CBF metrics, namely the available descent distance, relative velocity and Safety Variable (H_0) from the time when the MPC node is started

In some cases, the UAV landed on the first attempt because the safety conditions remained satisfied throughout the descent. When the safety conditions can no longer be recovered, i.e. $H_0 < 0$ and $\dot{H} + \gamma H < 0$, the optimization fails, the UAV transitions to GO_BACK, and landing is attempted again, as shown in Cases 4 (Fig. 2.12)

and 11 (Fig. 2.13). The results also show that embedding the barrier condition inside the optimiser improves the generated setpoints even under degraded predictions. Case 11 is the only one in which multiple optimisation failures occurred in a single experiment, with seven unsuccessful landing attempts before the final touchdown.

For completeness, the full landing video corresponding to Case 4 are provided in [44]. Figure 2.14 shows six sequential visual snapshots from Case 4.

2.7 Summary

This chapter developed the heave-landing pipeline as a timing-critical vertical-landing problem rather than as a purely geometric tracking problem. It showed how future deck motion is predicted, how a favourable touchdown target is selected from that prediction, how the descent is generated by a receding-horizon optimiser, and how a barrier-based safety layer prevents unsafe commitment. The experimental section demonstrated these components under both Simulation and hardware vision-driven operation, including cases with abort-and-retry behaviour. Together, these results establish the predictive and safety-critical landing framework that motivates the later RL-based moving-deck landing study.

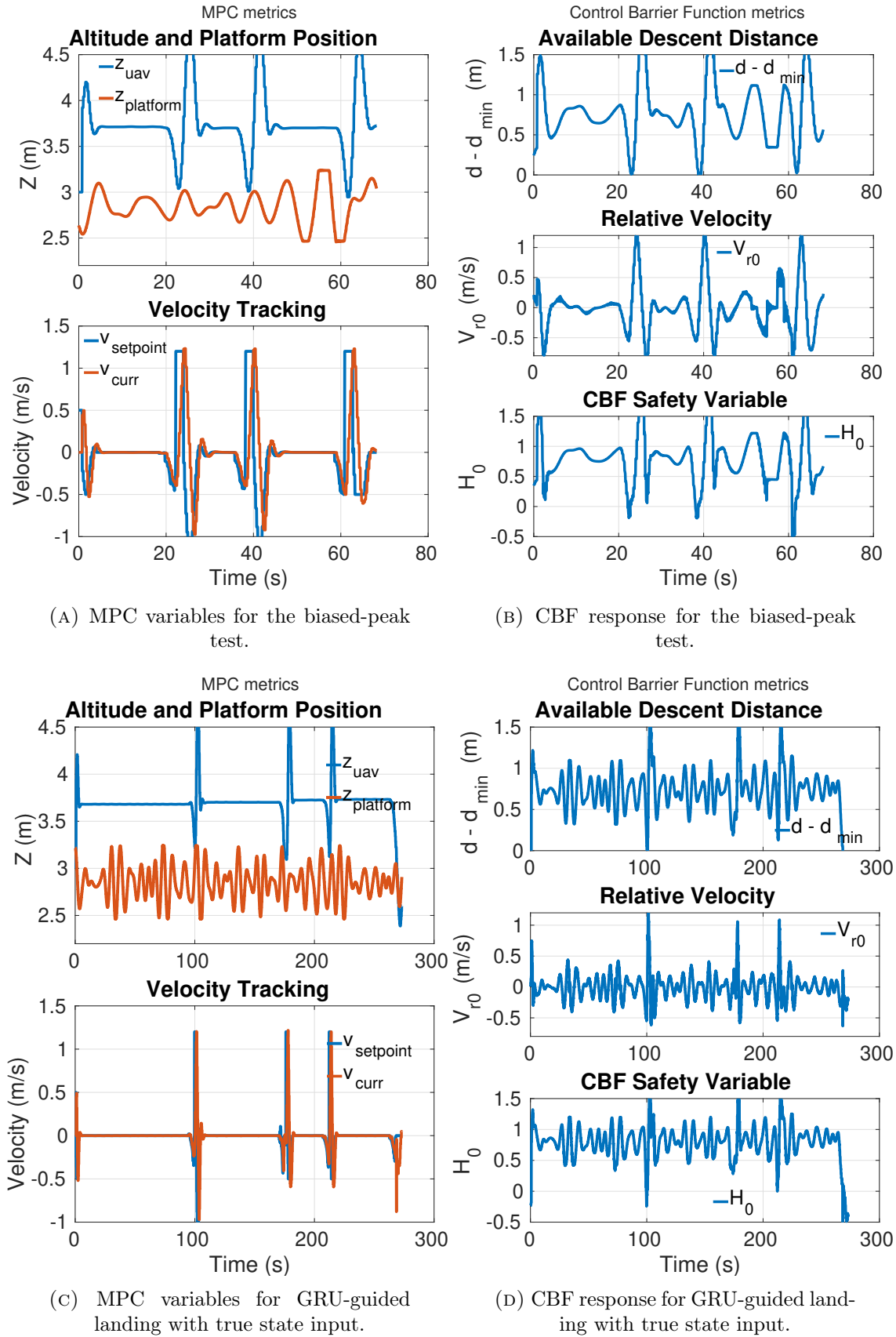


FIGURE 2.10: Simulation validation of the vertical landing pipeline. The top row shows the deliberately biased touchdown-target case and the bottom row shows a successful GRU-guided landing with a touchdown velocity of -0.22 m/s.

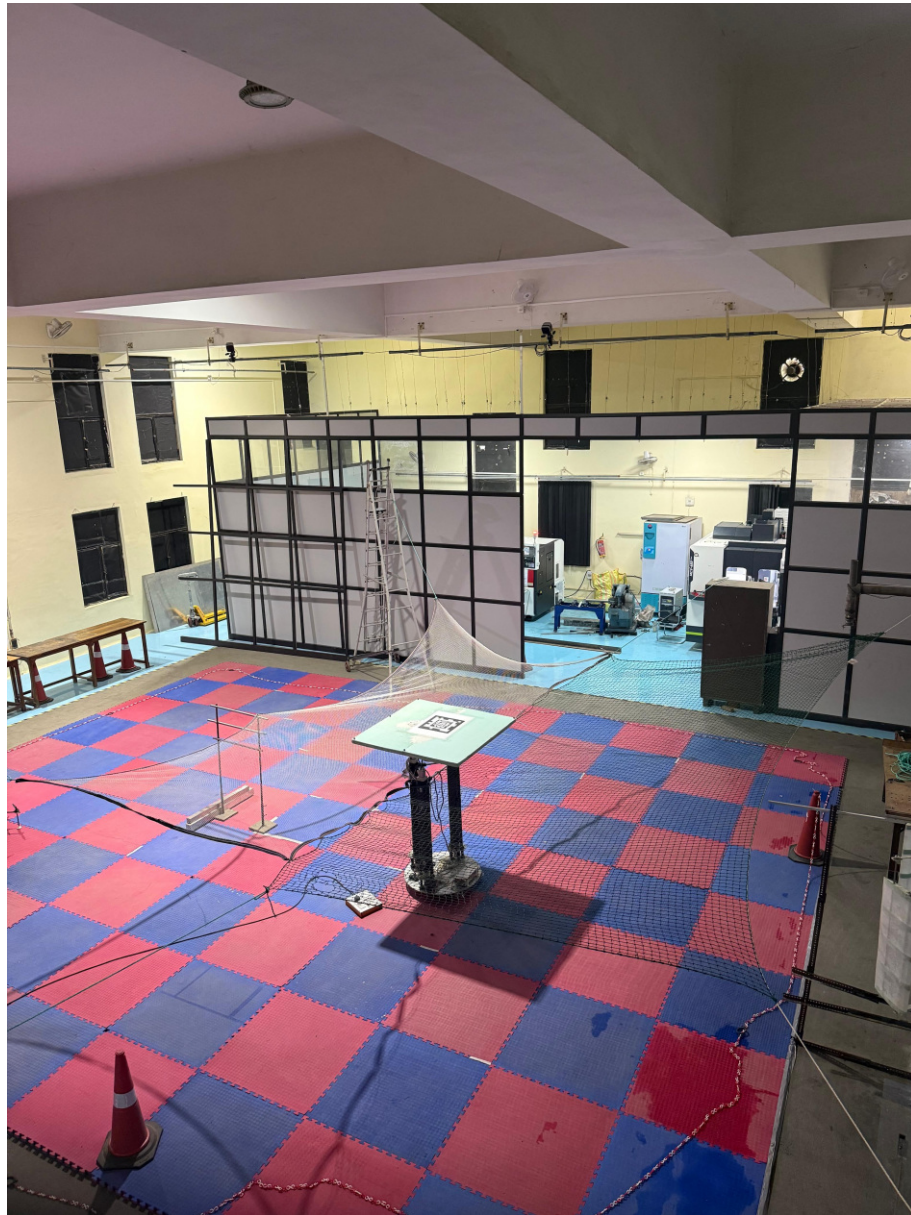


FIGURE 2.11: Experiment Setup with the Ship Deck Emulator

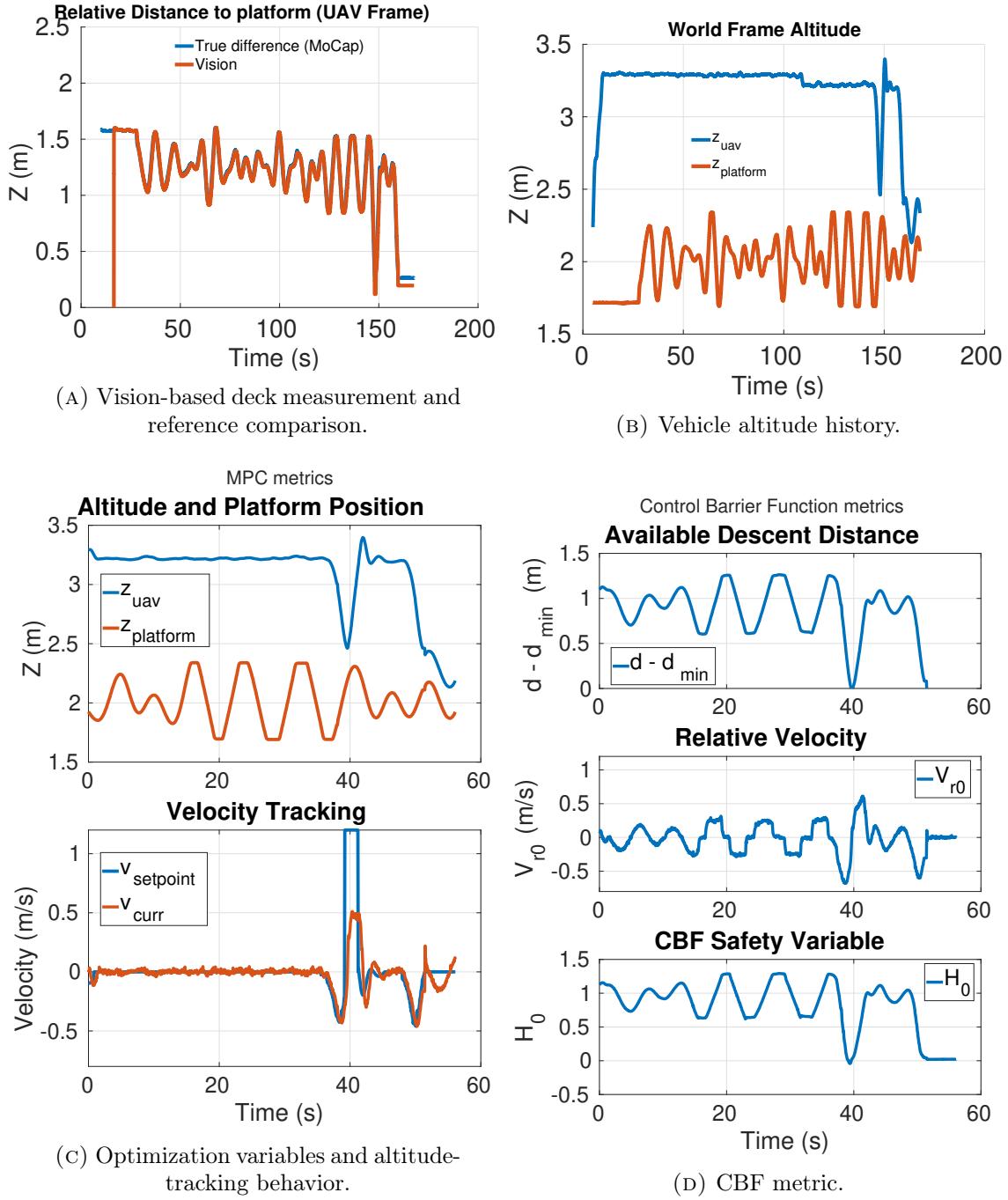


FIGURE 2.12: Vision-based landing, Case 4.

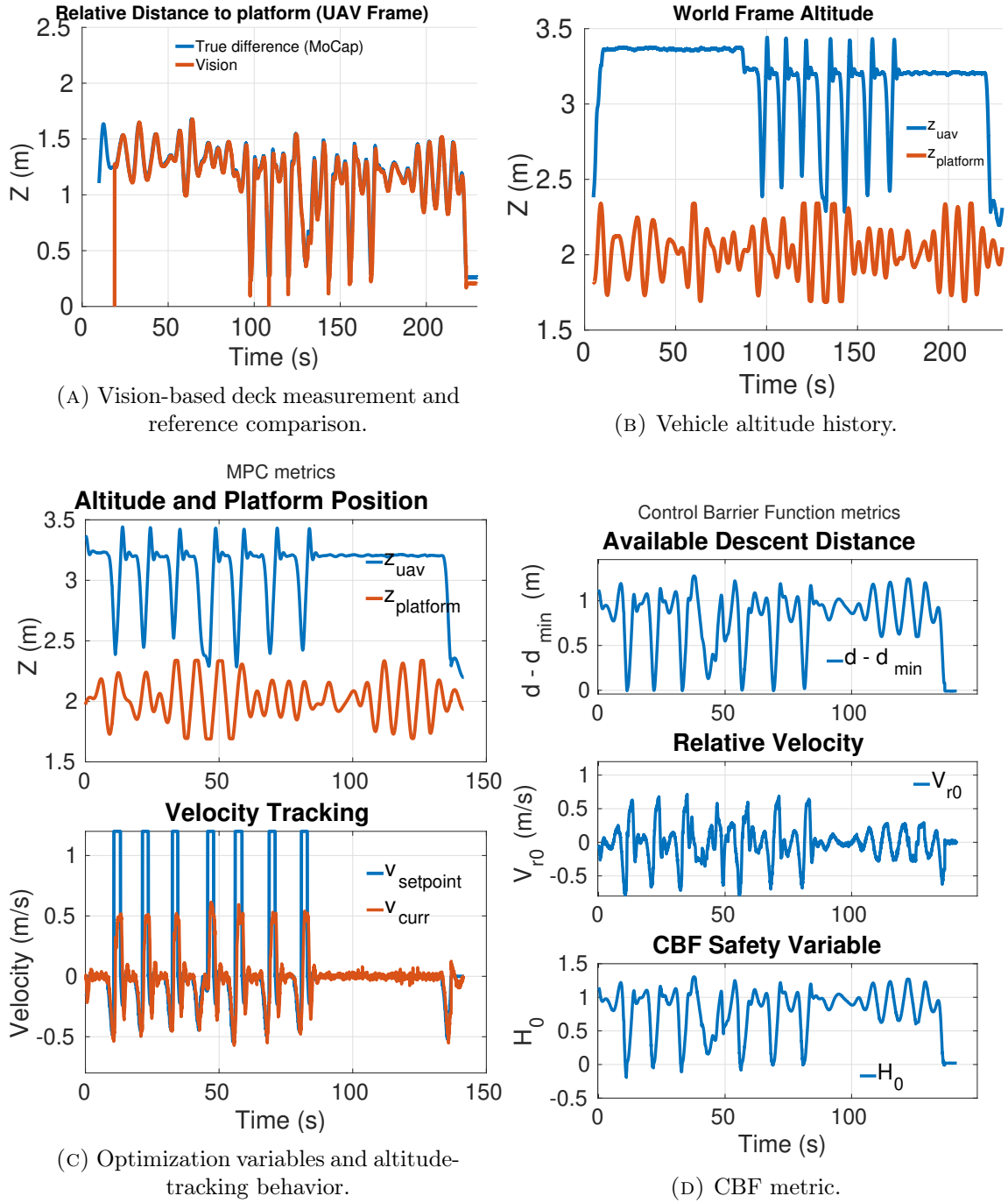
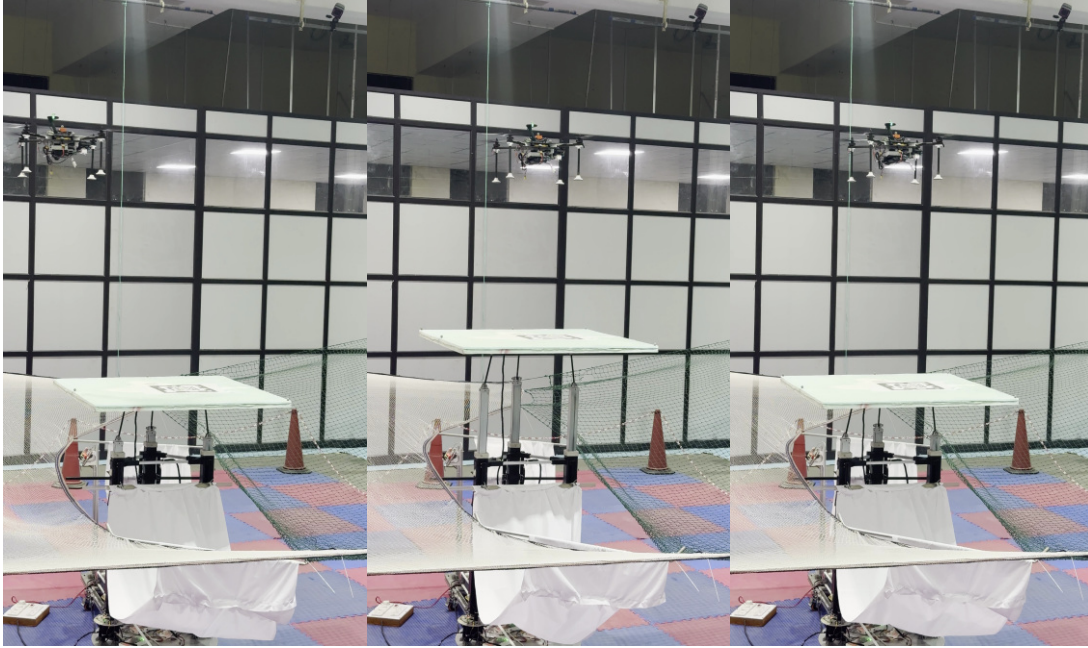


FIGURE 2.13: Vision-based landing, Case 11.



(A) Quadcopter UAV approaches the platform .

(B) Onset of vision-based position estimation.

(C) Onset of GRU-based motion prediction.



(D) Completion of the 40 s prediction window.

(E) Tracking of the MPC-generated set-points.

(F) Touchdown at a vertical velocity of -0.31 m/s.

FIGURE 2.14: Landing sequence for Case 4.

Chapter 3

Development of a PX4-compatible Reinforcement Learning Training Stack

3.1 Introduction

This chapter addresses the reinforcement-learning component of the thesis from a systems perspective. The goal is not to argue that RL alone solves autonomous landing, but to show how a policy-learning pipeline can be constructed so that it remains compatible with practical flight controllers. Reinforcement learning has already demonstrated strong performance in demanding aerial tasks such as agile racing, high-speed waypoint flight, and perception-aware navigation [45–47]. The unresolved difficulty is more operational: how to train such policies efficiently, adapt them to a realistic autopilot loop, and preserve an interface that can be validated and deployed through widely used flight stacks such as PX4 [48].

Accordingly, the chapter is organised around the training stack rather than around a single learning algorithm. It first motivates why RL is relevant for aerial landing and clarifies why strong published RL performance does not automatically translate into reusable autopilot-compatible workflows. It then defines the practical scope of the stack and the system requirements used to shape its design. The two core technical sections present the vectorised PX4-like pretraining stage and the lower-throughput PX4 SITL fine-tuning stage, with emphasis on the shared policy contract across both. The chapter closes with the transfer-validation and deployment pipeline used to test exported policies in simulation and prepare them for external execution. The implementation discussed throughout this chapter is provided through the open-source `uav_rl` software repository [49].

3.2 Why Reinforcement Learning for Aerial Robotics

3.2.1 Limitations of Classical Controllers in Complex Landing Tasks

Classical controllers remain the standard choice for many quadrotor applications because they are interpretable, efficient, and well aligned with flight-stack design. However, tasks such as landing on a moving platform expose the limitations of fixed-structure control more clearly than hover or waypoint tracking. In ship-deck landing, the vehicle must track a moving deck, reduce relative motion before contact, choose when to descend, and absorb the fact that touchdown quality depends on both kinematics and timing. A conventional controller can be engineered to solve parts of this problem, but it usually requires careful decomposition into phases, gain

scheduling, heuristics for descent initiation, and ad hoc rules for touchdown and termination.

This difficulty becomes more pronounced when the platform motion is not static or periodic in a trivial sense. If the deck translates, tilts, or heaves, the controller must handle coupled horizontal tracking, vertical regulation, and attitude stabilisation under tight timing constraints. This is precisely the regime in which RL becomes attractive: it allows the policy to optimise behaviour directly against task-level success and failure signals instead of relying only on manually sequenced control logic. Recent work from the Robotics and Perception Group at the University of Zurich has already shown that learned quadrotor policies can achieve champion-level autonomous racing [45], strong real-world agile-flight performance over 45 km/h [46], and perception-aware flight in clutter with a higher success rate and lower online computational burden than planning-heavy baselines [47]. It is also shown that acceleration-based and velocity-based action spaces show little performance degradation when the models of the underlying dynamics change [46]. Albeit, the acceleration-based action space is more agile for drone racing. We adopt a velocity-based action space for reasons of transfer and integration, not because it is the most agile parametrisation. In contrast to thrust-and-body-rate control, which is better suited to aggressive manoeuvres, velocity commands exploit the flight controller’s existing state estimator and inner loops, allowing the learned policy to focus on landing-relevant decisions such as relative tracking and descent timing [46].

3.2.2 Why RL Has Become Important in Robotics

Reinforcement learning has become important in robotics because it provides a way to learn control policies directly from interaction, especially in settings where the

environment is nonlinear, the dynamics are coupled, and the desired behavior is easier to specify through objectives than through explicit trajectories or switching logic. In recent years, RL has been especially influential in domains such as legged locomotion, bipedal control, agile manoeuvres, and contact-rich interaction. The core lesson from these domains is that RL can discover policies that coordinate multiple control objectives simultaneously when a hand-designed decomposition is either brittle or unnecessarily restrictive.

For aerial robots, this matters because quadrotors are underactuated, strongly coupled, and often required to operate under disturbances, moving references, and limited touchdown margins. In such settings, the policy must decide not only how to track a target, but also when to remain conservative and when to commit to descent or contact. This makes RL a reasonable candidate for extending beyond stabilisation and into higher-level landing behaviour.

3.2.3 High-Performance RL Exists, but Reusable Flight-Stack-Compatible Pipelines Remain Scarce

The cited aerial-RL literature already demonstrates that learned policies can achieve very strong performance. The practical difficulty lies elsewhere: most published results are not delivered as reusable open-source training stacks centred on controller-compatible command spaces and widely used autopilots such as PX4 or ArduPilot. A policy trained directly on simulator-specific forces or torques may perform well in a tightly matched environment, but the command interface itself can become the main obstacle during deployment. By contrast, PX4 [48] and similar autopilots rely on cascaded control structures and well-defined offboard command semantics that are robust across airframes, even if they still require controller tuning.

This report, therefore, does not treat RL as a stand-alone replacement for classical flight control. Instead, RL is used together with a system design that constrains the policy interface to velocity and yaw-rate commands, preserves a PX4-like cascaded backend during high-throughput training, and introduces an explicit SITL adaptation stage before deployment. The objective is not to replace cascaded flight controllers, but to combine their stability and compatibility with the task-level decision making that RL can learn.

3.3 Practical Gap and Scope of This Work

The practical gap addressed by this work is the lack of a ready-to-use RL training workflow that simultaneously offers the following properties:

- a vectorised simulator backend suitable for large-batch policy learning,
- a command interface (velocity in this case) that is meaningful for PX4 deployment,
- transfer tooling for execution outside the training environment.

The contribution of this chapter is therefore practical rather than algorithmic: an open-source training-and-transfer workflow in which a quadrotor policy is first pre-trained in a vectorized PX4-like environment and then fine-tuned in PX4 SITL before deployment-oriented transfer. This workflow is then studied in the context of a ship-deck sway-landing task.

3.4 Design Objectives and System Requirements

The stack was designed around four practical requirements. Figure 3.1 summarises the full two-stage training and transfer pipeline used in this work.

- **High data throughput during early learning:** the initial policy must be trained in a highly vectorised environment, since learning the coarse structure of quadrotor behaviour is sample-intensive.
- **Compatibility with PX4 deployment semantics:** the learned policy should not depend on a simulator-only actuation interface that disappears during deployment.
- **Stable checkpoint transfer between stages:** the policy observation and action contracts should remain consistent while the backend control implementation changes.
- **Operational transfer tooling:** the repository should provide policy playback, checkpoint evaluation, and a bridge from trained policies to external flight stacks and motion-capture-based execution.

At the software level, these requirements are organised into three layers: training tasks, PPO utilities, and transfer tools. The main components are summarised in Table 3.1.

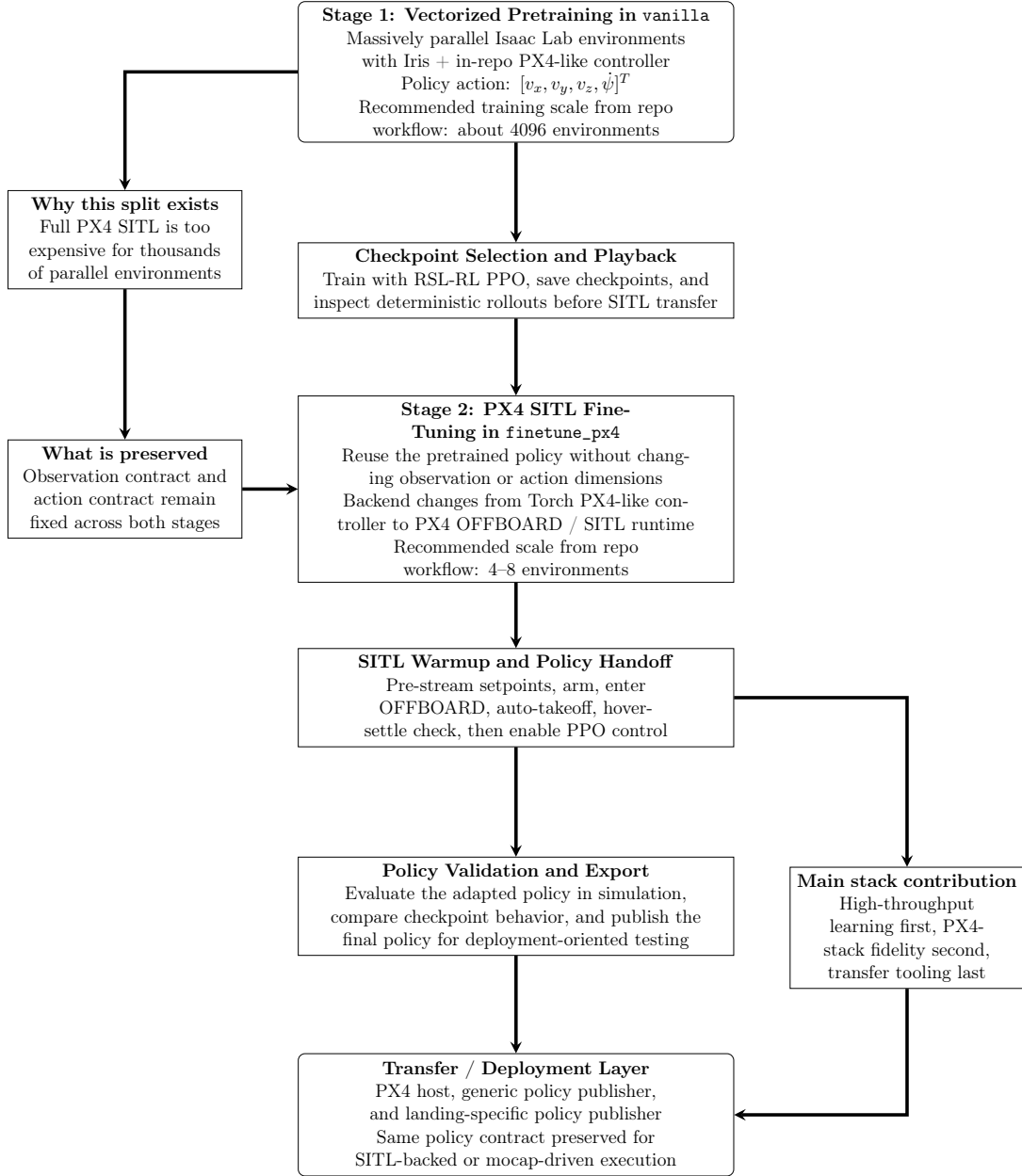


FIGURE 3.1: Overall two-stage RL training and transfer stack implemented in this work.

TABLE 3.1: Main components of the proposed RL stack.

Layer	Purpose
Pretraining	Vectorized RL training with an in-repo Torch PX4-like cascaded controller
Fine-tuning	Low-throughput RL training with commands routed through PX4 SITL OFFBOARD control
Evaluation	Checkpoint playback and rollout inspection without exploration noise
Transfer	External policy execution against PX4-compatible or mocap-driven pipelines

3.5 Stage 1: Vectorised Pretraining in a PX4-Like Environment

3.5.1 Motivation for the Pretraining Stage

The first stage addresses the throughput requirement of RL. Directly training against full PX4 SITL from the start is computationally impractical because each environment would need its own autopilot process and backend bridge. In contrast, the vectorized pretraining task uses a fully in-repo, Torch-based PX4-like control pipeline embedded inside the Isaac Lab action term, which makes it possible to train thousands of environments in parallel on a single simulator instance.

3.5.2 Environment Contract Used During Pretraining

The pretraining task keeps the policy interface intentionally compact.

- **Robot model:** Iris quadrotor.
- **Scene:** flat ground plane, moving platform, dome light, and contact sensing on robot bodies.

- **Policy action:** a 4-dimensional velocity command

$$\mathbf{u}_{\text{raw}} = [v_x, v_y, v_z, \dot{\psi}]^T \quad (3.1)$$

where v_x , v_y , and v_z are velocity commands in the world frame and $\dot{\psi}$ is yaw-rate command.

- **Command processing:** the raw action is scaled, offset, and clipped before it enters the controller backend,

$$\mathbf{u}_{\text{sp}} = \text{clip}(\mathbf{u}_{\text{raw}} \odot \mathbf{s} + \mathbf{o}) \quad (3.2)$$

where $\mathbf{s}$ and $\mathbf{o}$ denote per-channel scale and offset.

- **Policy rate:** 25 Hz.
- **Physics rate:** 250 Hz.

The pretraining environment uses `sim.dt = 1/250` and `decimation = 10`. Therefore, the PPO policy outputs a new command every ten physics steps, while the low-level controller is evaluated at every physics tick.

3.5.3 Embedded PX4-Like Control Pipeline

The main reason this stage remains useful for transfer is that the policy is not trained against arbitrary force commands. Instead, the action term implements a PX4-style cascaded structure:

- (1) velocity command to acceleration setpoint,
- (2) acceleration plus yaw to desired thrust and attitude,

- (3) attitude controller to body-rate setpoint,
- (4) body-rate controller to torque setpoint,
- (5) rotor allocation to motor angular velocities,
- (6) motor model and wrench application inside Isaac Sim.

This structure preserves the control semantics of a practical multicopter stack while still remaining efficient for simulation-side execution.

The exact controller sequence follows the same logic shown in Fig. 3.2. First, the policy command is converted to a controller setpoint,

$$\mathbf{u}_{\text{raw}} = [v_x, v_y, v_z, \dot{\psi}]^T, \quad (3.3)$$

$$\mathbf{u}_{\text{sp}} = \text{clip}(\mathbf{u}_{\text{raw}} \odot \mathbf{s} + \mathbf{o}). \quad (3.4)$$

The velocity loop then computes the acceleration setpoint through

$$\mathbf{e}_v = \mathbf{v}_{\text{sp}} - \mathbf{v}, \quad (3.5)$$

$$\mathbf{a}_{\text{sp}} = \mathbf{a}_{\text{ff}} + K_p^v \mathbf{e}_v + K_i^v \int \mathbf{e}_v dt + K_d^v \frac{d\mathbf{e}_v}{dt}, \quad (3.6)$$

with $\mathbf{a}_{\text{sp},xy}$ and $\mathbf{a}_{\text{sp},z}$ limited by configured acceleration bounds. The acceleration setpoint is converted to a force setpoint according to

$$\mathbf{F}_{\text{sp}} = m(\mathbf{a}_{\text{sp}} + g\mathbf{e}_z), \quad (3.7)$$

with the collective thrust command

$$T_{\text{sp}} = \|\mathbf{F}_{\text{sp}}\|, \quad (3.8)$$

clamped to thrust limits. The desired body z -axis is aligned with $\mathbf{F}_{\text{sp}}$, subject to the configured tilt bound, and the desired attitude quaternion $\mathbf{q}_{\text{sp}}$ is built from the desired body axes and yaw setpoint.

The attitude loop computes an orientation error

$$\mathbf{e}_R = \frac{1}{2} \text{vee}(R_{\text{sp}}^T R - R^T R_{\text{sp}}), \quad (3.9)$$

and the desired body-rate command is then

$$\boldsymbol{\omega}_{\text{sp}} = -K_p^{\text{att}} \mathbf{e}_R, \quad (3.10)$$

with the yaw-rate setpoint added on the third axis. The rate loop uses

$$\mathbf{e}_\omega = \boldsymbol{\omega}_{\text{sp}} - \boldsymbol{\omega}, \quad (3.11)$$

$$\boldsymbol{\tau}_{\text{sp}} = K_p^\omega \mathbf{e}_\omega + K_i^\omega \int \mathbf{e}_\omega dt - K_d^\omega \frac{d\boldsymbol{\omega}}{dt}. \quad (3.12)$$

Rotor allocation is then performed by constructing an allocation matrix A from the rotor geometry and thrust constants,

$$[T_{\text{sp}}, \tau_x, \tau_y, \tau_z]^T = A \boldsymbol{\omega}^2, \quad (3.13)$$

so that

$$\boldsymbol{\omega}^2 = A^+[T_{\text{sp}}, \boldsymbol{\tau}_{\text{sp}}]^T, \quad (3.14)$$

followed by clamping and

$$\boldsymbol{\omega} = \sqrt{\max(\boldsymbol{\omega}^2, 0)}. \quad (3.15)$$

The HIL-style actuator mapping used in the repository is

$$\mathbf{u}_{\text{hil}} = \text{clip} \left(\frac{\boldsymbol{\omega} - \mathbf{z}_{\text{armed}}}{\mathbf{s}_{\omega}} - \mathbf{o}_{\omega} \right), \quad (3.16)$$

with the inverse mapping used whenever rotor speed must be reconstructed from the HIL control representation. Finally, the motor model converts rotor speed to wrench terms using per-rotor thrust

$$F_i = k_i \omega_i^2, \quad (3.17)$$

rolling moment

$$\tau_{\text{roll},z} = \sum_i c_i \text{rot_dir}_i \omega_i^2, \quad (3.18)$$

and body drag

$$\mathbf{F}_{\text{drag}} = -C_{\text{drag}} \odot \mathbf{v}_{\text{body}}. \quad (3.19)$$

These forces and torques are the quantities actually applied inside Isaac Sim.

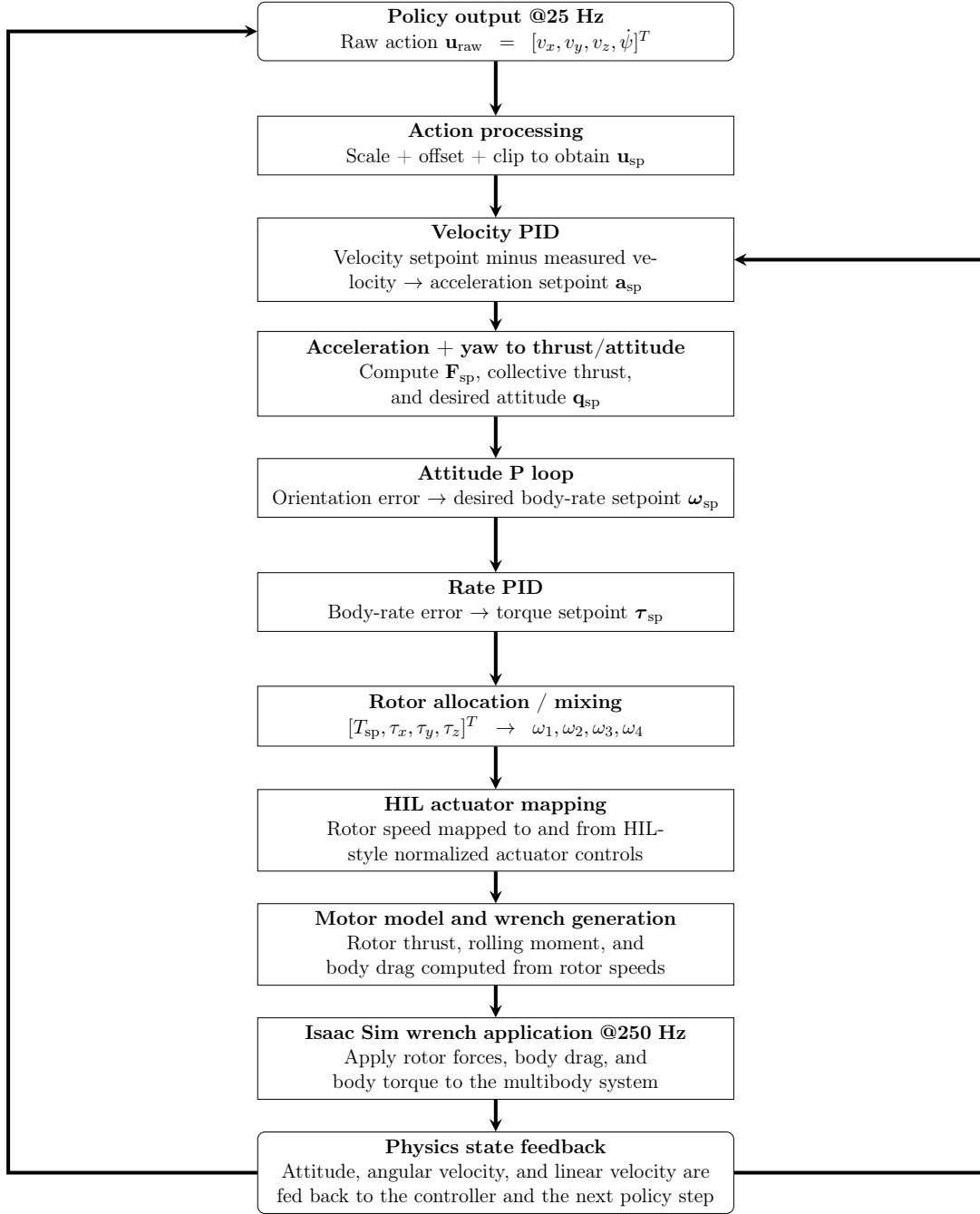


FIGURE 3.2: PX4-like cascaded controller used during vectorized pretraining, reproduced from the control/data flow used in the `uav_rl` repository.

3.5.4 Why Full SITL Is Not Used in Stage 1

The pretraining stage is intentionally not based on full PX4 SITL. The reason is not algorithmic but computational. Running a complete PX4 process, MAVLink bridge, and associated sensor pipeline for each environment becomes prohibitive once the environment count is scaled into the thousands, as all of these run on CPU. In that regime, throughput collapses, and PPO can no longer exploit one of the main advantages of simulation, namely, large-batch parallel rollout collection.

The PX4-like control backend, therefore, serves as a high-throughput approximation layer that keeps the policy interface close to deployment while avoiding the system’s overhead of thousands of full autopilot instances.

3.6 Stage 2: PX4 SITL Fine-Tuning

The second stage reuses the same policy contract but replaces the in-repo controller backend with a full PX4 SITL runtime. The critical point is that the policy action contract is preserved:

$$\mathbf{u} = [v_x, v_y, v_z, \dot{\psi}]^T. \quad (3.20)$$

Consequently, a policy trained in the vectorised stage can be resumed directly in the SITL stage without changing the observation shape or action semantics. What changes is only the backend that interprets the command. In the fine-tuning stage, the action term no longer computes motor thrust internally. Instead, the action is delivered to a PX4 runtime through an offboard velocity interface. The runtime maintains one PX4 SITL instance per environment together with the required backend state and simulated sensors, and returns the actuator reference that drives the simulated multirotor.

This stage is intentionally low throughput. The number of environments is reduced drastically, the rollout horizon is increased, and the update schedule is made more conservative because the policy entering this stage is already pretrained and the objective is no longer exploration at scale, but software-stack adaptation. The environment configuration is also tightened to make SITL operation reliable: the vehicle starts from a fixed pose, the platform is initialised at a known offset, platform motion is reduced relative to the generic vectorised stage, and domain randomisation is disabled in the SITL preset.

PPO does not begin collecting learning rollouts immediately after a simulator reset. The training script first steps zero actions until every SITL environment has completed its warmup sequence: connection establishment, offboard setpoint pre-stream, arming, mode switching, automatic takeoff, and hover settle. This avoids contaminating the learning signal with unstable takeoff transients and produces a better-defined start state for policy-controlled rollouts.

3.7 Transfer Validation and Deployment Pipeline

After training, checkpoints are first validated by exporting and replaying the learned policy through the transfer layer in simulation. This validation step uses deterministic policy execution rather than stochastic PPO sampling, so the trained command sequence can be inspected directly under the same velocity-and-yaw-rate contract used during training.

The repository also includes deployment-facing tools that preserve this contract outside the training environment. A PX4 host utility runs Pegasus Simulator together with PX4 and publishes the state topics required by the external bridge. A generic policy publisher reconstructs the observation required by the trained policy

and publishes the resulting command. A landing-specific publisher reconstructs the ship-deck landing observation vector, including the same projected-gravity convention and touchdown-clearance offset used during training.

On hardware, the learned policy is not treated as a stand-alone flight controller. Instead, it is inserted into a larger perception–estimation–flight-stack loop in which external sensing reconstructs the policy observation, the policy generates the same velocity and yaw-rate command contract used during training, and the downstream PX4-compatible stack remains responsible for low-level stabilisation and execution. This deployment-facing arrangement is summarised in Fig. 3.3.

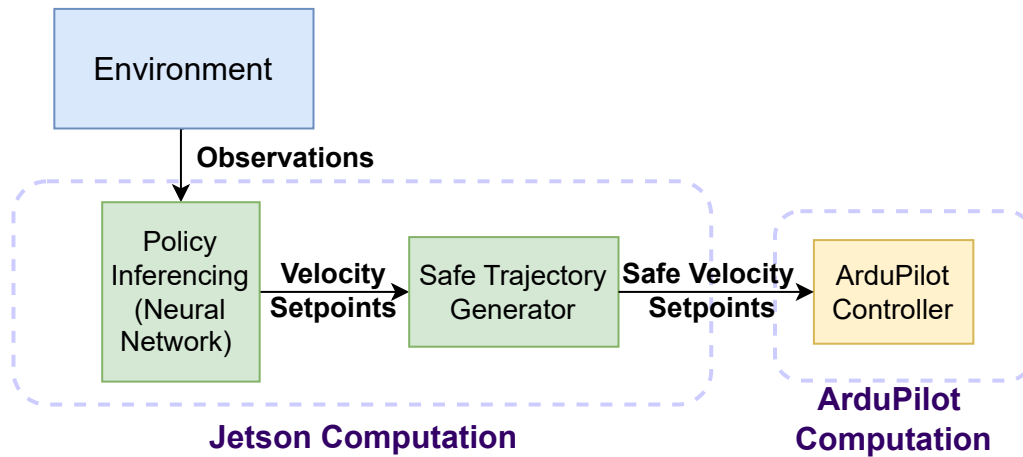


FIGURE 3.3: Deployment-side execution pipeline used after policy training.

3.8 Summary

This chapter described the RL training stack from the perspective of practical flight-stack compatibility. It explained why complex landing tasks motivate RL, why reusable open autopilot-compatible workflows remain scarce, and how those constraints shape the stack design. The resulting pipeline combines vectorised PX4-like pretraining for sample throughput with PX4 SITL fine-tuning for backend fidelity,

while preserving a shared velocity-command policy interface across both stages. The transfer layer then uses the same interface to validate exported policies in simulation and to prepare them for deployment-oriented execution.

Chapter 4

Landing on a Ship-Deck Emulator Using RL

This chapter documents the RL landing tasks used in the thesis and explains how the training stack of Chapter 3 is instantiated for moving-deck landing. The chapter is deliberately split into two parts. The first part describes the sway-landing task, in which the agent learns to land on a moving platform while maintaining lateral deck tracking, descent regulation, and touchdown quality. The second part describes the heave-landing extension, which retains the same deployable actor interface but replaces analytic platform motion with recorded ship-heave data, adds privileged critic observations, and introduces a recurrent GRU-based actor-critic variant [40].

4.1 Part I: Sway-Landing Task

This section discusses the problem setup of landing a UAV on a moving platform.

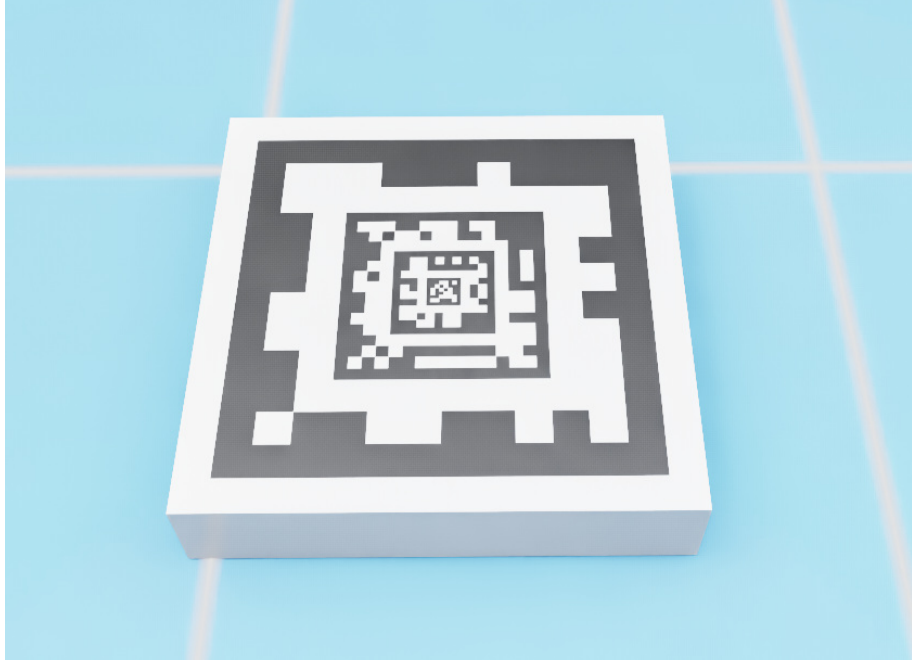


FIGURE 4.1: Moving landing platform used in the RL landing tasks.

4.1.1 Task Definition and Simulation Setup

The `landing_sway` task is a manager-based Isaac Lab environment in which each replicated environment contains the following main elements:

- an Iris quadrotor, shown in Fig. 4.2, and
- a landing platform, shown in Fig. 4.1.

The robot asset is the Iris configuration with landing legs. The landing platform is a rigid cuboid of size

$$(L_x, L_y, L_z) = (1.0, 1.0, 0.2) \text{ m}, \quad (4.1)$$

with initial position

$$\mathbf{p}_{\text{plat},0} = [0.0, 0.0, 0.1]^T. \quad (4.2)$$

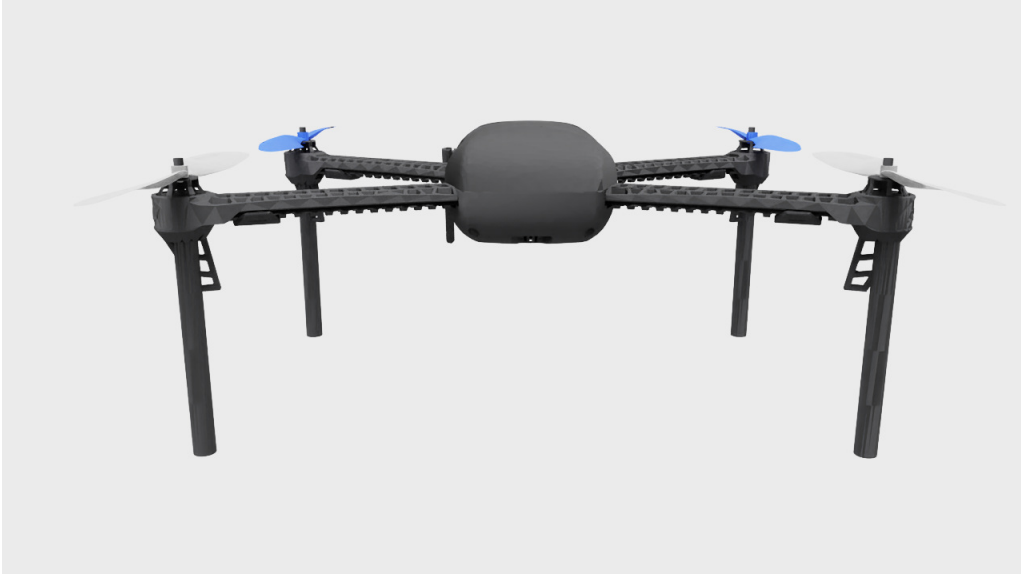


FIGURE 4.2: Iris UAV configuration with landing legs used during simulation training.

The baseline sway-policy configuration instantiates $N_{\text{env}} = 4098$ parallel environments with inter-environment spacing $d_{\text{env}} = 6.0$ m. This vectorised environment arrangement is illustrated in Fig. 4.3. The large number of parallel environments is central to the training strategy as the task is learned through high-throughput PPO rollouts [50] collected in parallel.

The low-level simulation runs with a time step

$$\Delta t_{\text{sim}} = 0.004 \text{ s} = \frac{1}{250} \text{ s}, \quad (4.3)$$

and the policy is evaluated every ten simulation steps, giving an effective policy period

$$\Delta t_{\pi} = 10 \Delta t_{\text{sim}} = 0.04 \text{ s}, \quad (4.4)$$

which corresponds to a policy frequency of 25 Hz. The episode timeout is $T_{\text{ep}} = 12.0$ s, which corresponds to 300 policy steps per episode.

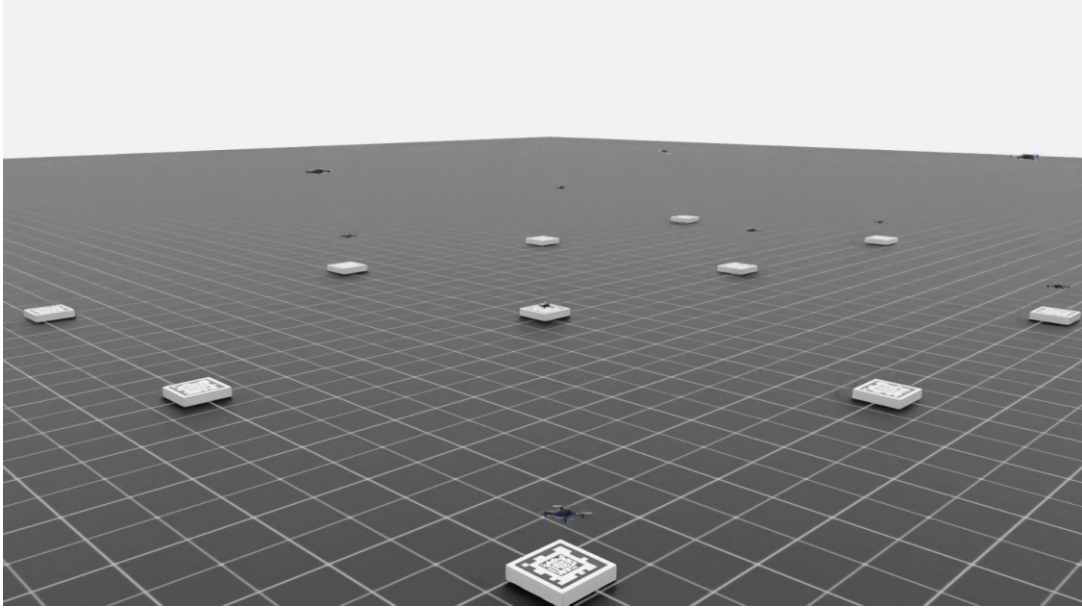


FIGURE 4.3: Parallel vectorised training of the sway-landing task in Isaac Lab.

Figure 4.4 shows the Isaac Lab scene used for the sway-landing task. The operational objective is to track the moving platform, regulate the vehicle attitude during approach, reduce the platform-relative vertical clearance, and terminate the episode through a smooth touchdown rather than through timeout, excessive tilt, altitude failure, or leaving the operating region.

4.1.2 Vehicle and Platform Models

The simulated robot asset is the Iris quadrotor shown in Fig. 4.2. One vehicle-specific geometric quantity is already explicit in the training configuration: $z_0 = 0.15$ m. This is the distance between the simulator body-frame origin and the lowest point of the landing gear. It is subtracted from the raw relative vertical position so that the policy observes a clearance-like variable rather than the height of the body origin itself.

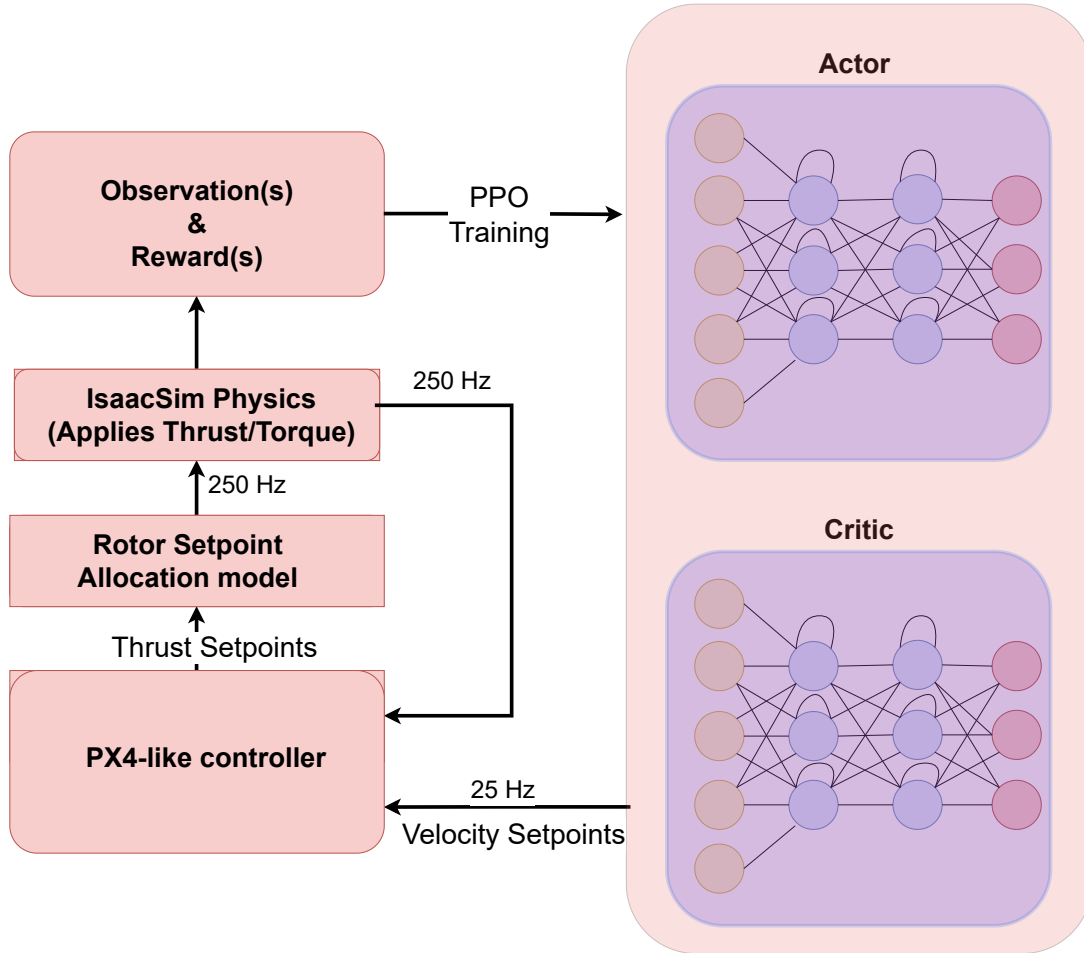


FIGURE 4.4: Isaac Lab environment used for the RL landing

The moving platform is a rigid kinematic body driven by the event term `MultiSinePlatformMotion`. In the baseline sway-policy setup, the active motion stage is `track_xy`, which excites only horizontal translation. Both decks x and y are synthesised from random superpositions of sinusoids with:

- 2 to 8 sinusoidal components per axis,
- per-component amplitude sampled in 0.4 m to 2.0 m,
- per-component frequency sampled in 0.2 Hz to 0.4 Hz,
- phase sampled uniformly in $[0, 2\pi]$, and

TABLE 4.1: Policy observation vector used in the baseline sway policy.

Observation term	Dimension	Description
$\mathbf{p}_{\text{rel}}$	3	World-frame relative position with z_0 correction
$\mathbf{v}_{\text{rel}}$	3	World-frame relative linear velocity
$\mathbf{q}_{\text{uav}}^w$	4	UAV root quaternion in world frame
$\boldsymbol{\omega}_{\text{rel}}$	3	World-frame relative angular velocity
$\mathbf{g}_b$	3	Projected gravity in the UAV body frame
$\mathbf{u}_{t-1}$	4	Previous action

- spectral decay factor equal to 1.0.

The resulting platform motion is clipped at the stage level so that

$$\|\mathbf{v}_{\text{plat}}\| \leq 1.0 \text{ m/s}, \quad \|\mathbf{a}_{\text{plat}}\| \leq 5.0 \text{ m/s}^2. \quad (4.5)$$

A stationary-environment probability of 0.3 is also applied, so that a fraction of the parallel environments remain static and the policy does not overfit to constantly moving decks.

4.1.3 Observation Space

The policy observation vector used in the sway task has dimension 20 and is assembled from the task observation definitions:

$$\mathbf{o}_{\text{sway}} = [\mathbf{p}_{\text{rel}}, \mathbf{v}_{\text{rel}}, \mathbf{q}_{\text{uav}}^w, \boldsymbol{\omega}_{\text{rel}}, \mathbf{g}_b, \mathbf{u}_{t-1}], \quad (4.6)$$

with dimensions $3 + 3 + 4 + 3 + 3 + 4 = 20$. The individual channels are summarised in Table 4.1.

The relative position is formed as

$$\mathbf{p}_{\text{rel}} = \mathbf{p}_{\text{uav}}^w - \mathbf{p}_{\text{plat}}^w, \quad (4.7)$$

and then corrected in the vertical channel as

$$\tilde{\mathbf{p}}_{\text{rel}} = \begin{bmatrix} x_{\text{rel}} \\ y_{\text{rel}} \\ z_{\text{rel}} - z_0 \end{bmatrix}. \quad (4.8)$$

The linear and angular velocity observations are both world-frame relative quantities:

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{uav}}^w - \mathbf{v}_{\text{plat}}^w, \quad \boldsymbol{\omega}_{\text{rel}} = \boldsymbol{\omega}_{\text{uav}}^w - \boldsymbol{\omega}_{\text{plat}}^w. \quad (4.9)$$

The quaternion is simply the Platform root quaternion in the world frame, since in the final implementation, the camera on the UAV would be stabilised via a gimbal.

The projected-gravity term is

$$\mathbf{g}_b = R_{wb}^\top \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}, \quad (4.10)$$

and the final channel is the previous action. For the sway task, the critic observation group is identical to the actor observation group, i.e. there are no privileged critic-only variables in the baseline sway policy.

4.1.4 Action Space and Command Semantics

The policy action is four-dimensional and represents commanded translational velocity and yaw rate:

$$\mathbf{u} = \begin{bmatrix} v_x & v_y & v_z & \dot{\psi} \end{bmatrix}^\top. \quad (4.11)$$

The translational components v_x , v_y , and v_z are interpreted in the world frame. They are passed into the PX4-like cascaded controller already described in Fig. 3.2, which converts the commanded velocity setpoint into acceleration, thrust, attitude, and rate requests before rotor allocation.

In the baseline sway policy, the processed action is clipped using

$$\mathbf{v}_{\min} = \begin{bmatrix} -0.8 & -0.8 & -0.8 \end{bmatrix}^\top \text{ m/s}, \quad (4.12)$$

$$\mathbf{v}_{\max} = \begin{bmatrix} 0.8 & 0.8 & 1.0 \end{bmatrix}^\top \text{ m/s}, \quad (4.13)$$

$$\dot{\psi}_{\min} = -35^\circ/\text{s}, \quad (4.14)$$

$$\dot{\psi}_{\max} = 35^\circ/\text{s}. \quad (4.15)$$

4.1.5 Reward Design for Sway Landing

The reward structure in `landing_sway` is dense enough to make platform tracking learnable, but sparse enough that touchdown quality remains the decisive event. Table 4.2 lists the grouped active reward components used in the base sway policy.

TABLE 4.2: Grouped active reward components in the base sway policy.

Reward group	Active weight(s)
Failure termination penalty	-500
Position tracking reward	10.0
Velocity tracking reward	+2.0 on horizontal deck-relative velocity matching
Vertical clearance penalty	-0.5, active only when platform-relative clearance exceeds 1.0 m
Action magnitude penalty	$v_x : -2.5, v_y : -2.5, v_z : -2.5, \dot{\psi} : -1.2$
Action rate penalty	$\Delta v_x : -50, \Delta v_y : -50, \Delta v_z : -35$
Attitude regularization	angular rate -0.5, angular-velocity rate -50, roll/pitch rate -1.0, yaw-rate error -10, yaw error -2, uprightness -20
Touchdown quality reward	1.0

The dominant dense shaping terms are:

- horizontal position tracking, implemented as

$$r_{\text{track}} = 1 - \tanh\left(\frac{\|\mathbf{p}_{\text{rel},xy}\|_2}{\sigma_{xy}}\right), \quad \sigma_{xy} = 2.0 \text{ m}, \quad (4.16)$$

- horizontal velocity tracking against the platform motion, implemented as

$$r_{\text{vel}} = 1 - \tanh\left(\frac{\|\mathbf{v}_{\text{rel},xy}\|_2}{\sigma_v}\right), \quad \sigma_v = 0.5 \text{ m/s}, \quad (4.17)$$

so that the policy is rewarded when the platform-relative horizontal velocity approaches zero, and

- one-sided vertical-clearance shaping,

$$r_{\text{clr}} = \max(0, z_c - z_{\text{th}}), \quad z_{\text{th}} = 1.0 \text{ m}, \quad (4.18)$$

with $z_c = z_{\text{rel}} - z_0$. Because the associated reward weight is negative, this term penalises the vehicle only when the platform-relative vertical clearance exceeds 1.0 m; below that threshold, no vertical-clearance penalty is applied.

The remaining shaping channels are regularisation terms. The speed penalties suppress unnecessary translational motion, the action-magnitude and action-smoothness penalties discourage aggressive command issuance, the acceleration penalty damps abrupt thrust-induced changes, and the grouped attitude-regularisation terms penalise body rates, rate oscillations, yaw misalignment, and loss of uprightness.

Touchdown is detected when the norm of the body contact force first exceeds

$$F_{\text{th}} = 2.0 \text{ N}. \quad (4.19)$$

The touchdown-quality term then evaluates the pre-impact state. In the sway policy, a touchdown is labelled good only if all of the following are satisfied:

$$|v_{\text{rel},z}^-| \leq 0.3 \text{ m/s}, \quad (4.20)$$

$$\|\mathbf{e}_{xy}^-\|_2 \leq 0.4 \text{ m}, \quad (4.21)$$

$$|\phi^-| \leq 12^\circ, \quad (4.22)$$

$$|\theta^-| \leq 12^\circ, \quad (4.23)$$

$$(4.24)$$

with the additional requirements that the touchdown point remain inside the deck footprint and the touchdown attitude remain inside the configured roll, pitch, and yaw limits. Good touchdowns receive a base reward

$$R_{\text{good}} = 5000, \quad (4.25)$$

bad touchdowns receive

$$R_{\text{bad}} = -100, \quad (4.26)$$

and a centre-proximity bonus

$$R_{\text{center}} = 1000 c_{\text{ctr}}, \quad c_{\text{ctr}} \in [0, 1], \quad (4.27)$$

is added to good touchdowns according to the normalised XY closeness factor $c_{\text{ctr}} \in [0, 1]$ relative to the platform centre.

The failure-penalty channel is triggered only by timeout, excessive attitude tilt, low-altitude crash, high-altitude escape, and horizontal out-of-bounds termination. Its base penalty value is -10 , scaled by reward weight 50.0 . A separate terminal-touchdown bonus is added when the episode ends through the intended touchdown event.

4.1.6 Termination Logic, Reset Distributions, and Domain Randomisation

The sway task uses the following termination conditions:

- **timeout:** episode length exceeds 12.0 s ,

- **attitude tilt:** either roll or pitch magnitude exceeds 30° ,
- **touchdown:** body contact-force threshold crossing,
- **low-altitude crash:** UAV height drops below 0.0 m,
- **high-altitude escape:** UAV height rises above 7.0 m, and
- **horizontal boundary exit:** horizontal distance from the environment origin exceeds 6.0 m.

Only touchdown is intended as a successful terminal event; all others contribute to the failure-penalty channel.

The baseline sway reset distribution samples the UAV pose and velocity from

$$x \in [-5.0, 5.0] \text{ m}, \quad y \in [-5.0, 5.0] \text{ m}, \quad z \in [0.1, 5.0] \text{ m}, \quad (4.28)$$

$$\phi \in [-0.2, 0.2] \text{ rad}, \quad \theta \in [-0.15, 0.15] \text{ rad}, \quad \psi \in [-\pi, \pi], \quad (4.29)$$

with translational and angular velocity perturbations sampled in

$$[-0.2, 0.2] \quad (4.30)$$

for x , y , z , roll, and pitch, and

$$\dot{\psi} \in [-0.5, 0.5] \text{ rad/s} \quad (4.31)$$

for yaw rate.

The active domain-randomisation stack is summarised in Table 4.3. These perturbations are injected not only into the physical model, but also into the command

pipeline and estimator-like observation channels, so that the policy is trained against both plant mismatch and delayed/noisy state information.

TABLE 4.3: Active domain-randomisation settings in the sway-landing task.

Randomization channel	Probability	Active parameter values
Mass perturbation	0.5	Gaussian std 0.1 kg, clip 0.3 kg
Action delay	0.3	Delay between 1 and 3 policy steps
State-estimation noise	0.5	Position 0.03 m, linear velocity 0.08 m/s, angular velocity 0.04 rad/s, attitude 0.015 rad, projected gravity 0.04
Thrust asymmetry	0.5	Multiplicative scale in [0.9, 1.1]
Motor lag	0.4	Time constant in [0.02, 0.08] s
Velocity-loop gain noise	0.3	K_P :std(0.15, 0.15, 0.35), K_I :std(0.05, 0.05, 0.20), K_D :std(0.03, 0.03, 0.05)

4.1.7 PPO Configuration for the Base and Fine-Tuned Sway Policies

The sway landing policy uses a feed-forward actor-critic trained with PPO [50], with identical observation groups for actor and critic. The base sway policy is the main training configuration, while the fine-tuned sway policy resumes from that base checkpoint and reduces the update aggressiveness for continuation training. Table 4.4 summarises the relevant PPO settings.

TABLE 4.4: PPO settings for the sway-landing policy and its continuation fine-tuning run.

Parameter	Base sway policy	Fine-tuned sway policy
Actor hidden dims	[128, 64]	[128, 64]
Critic hidden dims	[128, 128, 64]	[128, 128, 64]
Initial noise std	1.0	1.0
Noise parameterization	log std	log std
Steps per env	24	24
Max iterations	2000	1000
Learning rate	10^{-4}	10^{-5}
Entropy coefficient	10^{-2}	10^{-4}
Clip parameter	0.2	0.2
Learning epochs	5	5
Mini-batches	4	4
Desired KL	10^{-2}	10^{-3}
γ	0.99	0.99
λ	0.95	0.95
Initialization	From scratch	Continued from base sway policy

The practical interpretation is straightforward. The base sway policy is the main task-learning run. The fine-tuned sway policy keeps the same environment, observation contract, action space, and network architecture, but uses a conservative continuation schedule with a smaller learning rate, a smaller entropy coefficient, and a lower desired KL so that touchdown behaviour can be refined without large policy drift.

TABLE 4.5: Final simulation metrics for the fine-tuned sway policy.

Metric	Fine-tuned sway policy (%)
Touchdown terminations	93.96
Good touchdown rate over all episodes	91.23
Good touchdown percentage among touchdowns	96.72
Bad touchdown percentage among touchdowns	3.28
Timeout terminations	0.84
Attitude-tilt terminations	0.04
Out-of-bounds terminations	5.19
Crash-low terminations	0.00
Crash-high terminations	0.00

4.1.8 Sway-Landing Results

Table 4.5 summarises the final episode-end statistics of the fine-tuned sway policy.

The touchdown rate remains the main performance indicator, while the remaining termination channels reveal the residual lateral-tracking failure modes.

The fine-tuned sway policy, therefore, succeeds primarily through direct touchdown termination, with the remaining failures dominated by residual lateral boundary exits rather than by altitude or attitude violations.

4.1.8.1 Hardware Results

To illustrate the closed-loop landing behaviour beyond the aggregate curriculum metrics, Figs. 4.5–4.8 show landing performance on a deck emulator following sinusoidal motion with amplitudes 0.45 m and 0.55 m. For each amplitude, two forcing frequencies are considered: 0.1 Hz and 0.2 Hz. In all cases, the vehicle is first moved manually to a random initial position above the operating region, and the learned policy is then enabled at time $t = t_{\text{act}}$.

The four traces reported correspond to the lateral and vertical motion channels together with the attitude response. The 0.45 m cases in Figs. 4.5 and 4.6 show the same policy under a slightly milder deck excursion, while Figs. 4.7 and 4.8 retain the previously discussed 0.55 m deck motion. For completeness, the full landing video of these two cases is provided in [51] and [52], and snapshots for the latter are provided in Fig. 4.9.

4.1.9 Remarks

The RL policy generated agile setpoints for landing on a platform at a velocity of up to 0.7 m/s. Comparing this with the controller-design results of Chapter 2, the RL experiments showcase landing under faster periodic motion while preserving the same high-level objective of safe deck-relative touchdown. An important distinction is that the RL policy does not require a separately tuned tracking-and-descent logic before deployment; the landing behaviour is learned directly through the reward and termination structure.

4.2 Part II: Heave-Landing Extension

This section discusses the changes made to the sway environment to tackle heaving motion, which, as explained in Section 2.1, is more complicated to deal with due to the motion of the platform being on the same axis as the UAV's landing trajectory.

4.2.1 Why Heave Landing is a Structured Extension of the Sway Task

The `heave_landing` task is an extension of the sway-landing formulation. The same four-dimensional action space is preserved,

$$\mathbf{u} = [v_x, v_y, v_z, \dot{\psi}]^T, \quad (4.32)$$

and the actor observation interface is also preserved so that deployment remains feasible with the same estimator and command space. The key additions are elsewhere:

- the platform motion is no longer analytic XY sway, but replayed ship-heave data,
- the critic is given privileged future platform motion and exact velocity channels during training, and
- introduces an asymmetric recurrent GRU actor-critic for temporally structured deck motion.

This lets the heave task reuse the practical RL interface learned in sway landing while injecting the stronger temporal structure that is required when the vertical deck profile comes from recorded ship motion.

4.2.2 Heave dataset for RL

In the heave task, the moving platform is driven by the event term `CSVHeavePlatformMotion`.

The motion source is a normalised ship-heave SCONE dataset [35]. The playback settings in the post-initialisation configuration are:

- sample rate 20 Hz,
- amplitude scale factor 0.32,

Thus, unlike the sway task, the heave task is continuously exposed to dataset-driven vertical motion derived from ship-heave measurements [35].

4.2.3 Actor Observation and Privileged Critic Observation

The actor observation group in `heave_landing` is intentionally kept identical to the sway policy observation group. The actor still receives the same 20-dimensional vector built from platform-relative position, relative linear velocity, root quaternion, relative angular velocity, projected gravity, and previous action. The critic, however, is extended with privileged observations. In addition to the same 20 policy channels, the critic receives:

- the next 6 s of future platform world-frame z samples at 20 Hz, i.e. 120 privileged values,
- the exact UAV world-frame linear velocity, and
- the exact platform world-frame linear velocity.

This produces a critic observation dimension of

$$20 + 120 + 3 + 3 = 146. \quad (4.33)$$

The full actor-critic split is summarised in Table 4.6.

This asymmetric observation design is one of the main technical differences between the sway and heave tasks. The critic is allowed to exploit simulator privilege to assign

TABLE 4.6: Actor and critic observation structure in `heave_landing`.

Observation channel	Actor	Critic
Relative position with z_0 correction	3	3
Relative linear velocity	3	3
UAV root quaternion in world frame	4	4
Relative angular velocity	3	3
Projected gravity	3	3
Previous action	4	4
Future platform z trajectory (6 s at 20 Hz)	—	120
Exact UAV world linear velocity	—	3
Exact platform world linear velocity	—	3
Total dimension	20	146

values more accurately in a temporally structured landing problem, but the actor remains tied to the deployable observation contract. In a real system, the future-deck information used by the critic would need to be supplied by the prediction pipeline described in Chapter 3; during RL training, however, the simulator can provide it directly.

4.2.4 Reward and Touchdown Changes Relative to Sway Landing

The heave task reuses the same basic reward scaffold as `landing_sway`, but several values are deliberately changed. Table 4.7 lists the grouped active reward components used in the recurrent heave policy.

TABLE 4.7: Grouped active reward components in the recurrent heave policy.

Reward group	Active weight(s)
Failure termination penalty	50.0, applied to a base failure value of -10
Position tracking reward	1.0
Vertical clearance penalty	-0.5 , active only when platform-relative clearance exceeds 1.0 m
Action magnitude penalty	$v_x : -2.5, v_y : -2.5, v_z : -2.5, \dot{\psi} : -1.2$
Action rate penalty	$\Delta v_x : -40, \Delta v_y : -40, \Delta v_z : -20$
Attitude regularization	angular rate -0.5 , angular-velocity rate -50 , roll/pitch rate -1.0 , yaw-rate error -10 , yaw error -2 , uprightness -20
Touchdown quality reward	1.0

The interpretation is that heave landing is less about aggressive XY pursuit and more about timing-sensitive vertical contact. Accordingly, the position-tracking reward is reduced from 10.0 to 1.0, the velocity-tracking reward is disabled, and the action-smoothness penalties in x , y , and especially z are softened. The touchdown quality test is also stricter: the acceptable pre-impact vertical speed is reduced from 0.30 m/s to 0.25 m/s, while bad touchdowns receive a larger penalty of

$$R_{\text{bad}} = -500. \quad (4.34)$$

The good-touchdown reward remains 5000, and the centre-proximity bonus remains 1000, so the heave task shifts the balance by making mistimed landings more expensive rather than by inflating the success reward.

The heave task also uses a much tighter reset envelope around the platform:

$$x \in [-0.3, 0.3] \text{ m}, \quad y \in [-0.3, 0.3] \text{ m}, \quad z \in [2.5, 4.0] \text{ m}, \quad (4.35)$$

$$\phi \in [-0.2, 0.2] \text{ rad}, \quad \theta \in [-0.15, 0.15] \text{ rad}, \quad \psi \in \left[-\frac{\pi}{10}, \frac{\pi}{10}\right], \quad (4.36)$$

with the same narrow velocity perturbation ranges as the sway task. This reset change reflects the role of the heave extension: it isolates vertical synchronisation on top of already-localised deck alignment, rather than re-solving large horizontal approach errors from scratch.

4.2.5 Privileged-Critic Recurrent GRU Policies

The recurrent GRU variant [40] is the extension that introduces temporal memory beyond the instantaneous actor observation vector. In the recurrent configuration, the actor and critic use asymmetric recurrent stacks. The custom recurrent actor-critic uses a `StackedMemory` module that chains sequential one-layer GRU blocks with potentially different hidden sizes for actor and critic. This allows the actor and critic to have different recurrent capacities while still sharing the same high-level recurrent interface. The feed-forward and recurrent runner settings are listed in Table 4.8.

The GRU variant keeps the same actor action interface,

$$\mathbf{u} = [v_x, v_y, v_z, \dot{\psi}]^T, \quad (4.37)$$

but augments the policy with temporal memory across rollout steps. The actor GRU stack is [128, 64], while the critic GRU stack is much larger at [512, 256]. This

TABLE 4.8: PPO runner configurations defined for `heave_landing`.

Parameter	GRU recurrent variant
Actor obs group	policy
Critic obs group	critic
Actor MLP hidden dims	[32]
Critic MLP hidden dims	[128]
Actor recurrent stack	[128, 64] GRU
Critic recurrent stack	[512, 256] GRU
Final Actor stack	GRU[128, 64] + MLP[32]
Final Critic stack	GRU[512, 256] + MLP[128]
Steps per env	48
Max iterations	4000
Learning rate	10^{-7}
Entropy coefficient	10^{-4}
Learning epochs	5
Mini-batches	6
Desired KL	10^{-3}
Noise parameterization	log std
Initial noise std	1.0

asymmetry is deliberate: the critic must exploit a higher-dimensional privileged observation vector and assign value over temporally structured ship-heave trajectories, whereas the actor must remain compact enough to preserve a practical deployment path.

The recurrent PPO configuration also uses a much more conservative optimisation schedule than the sway policy: smaller entropy coefficient, lower desired KL, longer rollout windows, and a very small learning rate. This is consistent with the role of the heave task. Once future platform motion is provided to the critic and the actor

TABLE 4.9: Final simulation metrics for the recurrent heave policy.

Metric	Recurrent heave policy (%)
Touchdown terminations	98.61
Good touchdown rate over all episodes	91.25
Good touchdown percentage among touchdowns	92.47
Bad touchdown percentage among touchdowns	7.53
Timeout terminations	0.43
Attitude-tilt terminations	0.99
Out-of-bounds terminations	0.00
Crash-low terminations	0.00
Crash-high terminations	0.00

is equipped with recurrent memory, training emphasis shifts away from aggressive exploration and toward stable refinement of contact timing and descent synchronisation.

4.2.6 Heave-Landing Results

The final simulated landing statistics of the recurrent heave policy are summarised in Table 4.9.

The recurrent heave policy almost eliminates horizontal boundary failures and reaches a high touchdown-termination rate, but it still incurs a small fraction of tilt-related failures because the timing-sensitive vertical landing problem is more tightly coupled to transient attitude excursions near contact. The hardware experiments for this heave-targetted policy are still ongoing.

4.2.7 Summary of the RL Landing Chapter

This chapter showed how the RL stack is specialised into two related landing tasks. The sway task establishes the base formulation: a high-throughput vectorised environment with world-frame velocity commands, structured reward shaping, randomised dynamics, and touchdown-quality-driven termination statistics. The heave task then extends that formulation by replacing analytic deck motion with recorded ship-heave traces, adding privileged critic information, and introducing a recurrent GRU policy for temporally structured landing. Together, the two tasks show how the same deployment-oriented action interface can support both generic moving-deck landing and more timing-sensitive landing on recorded ship-motion data.

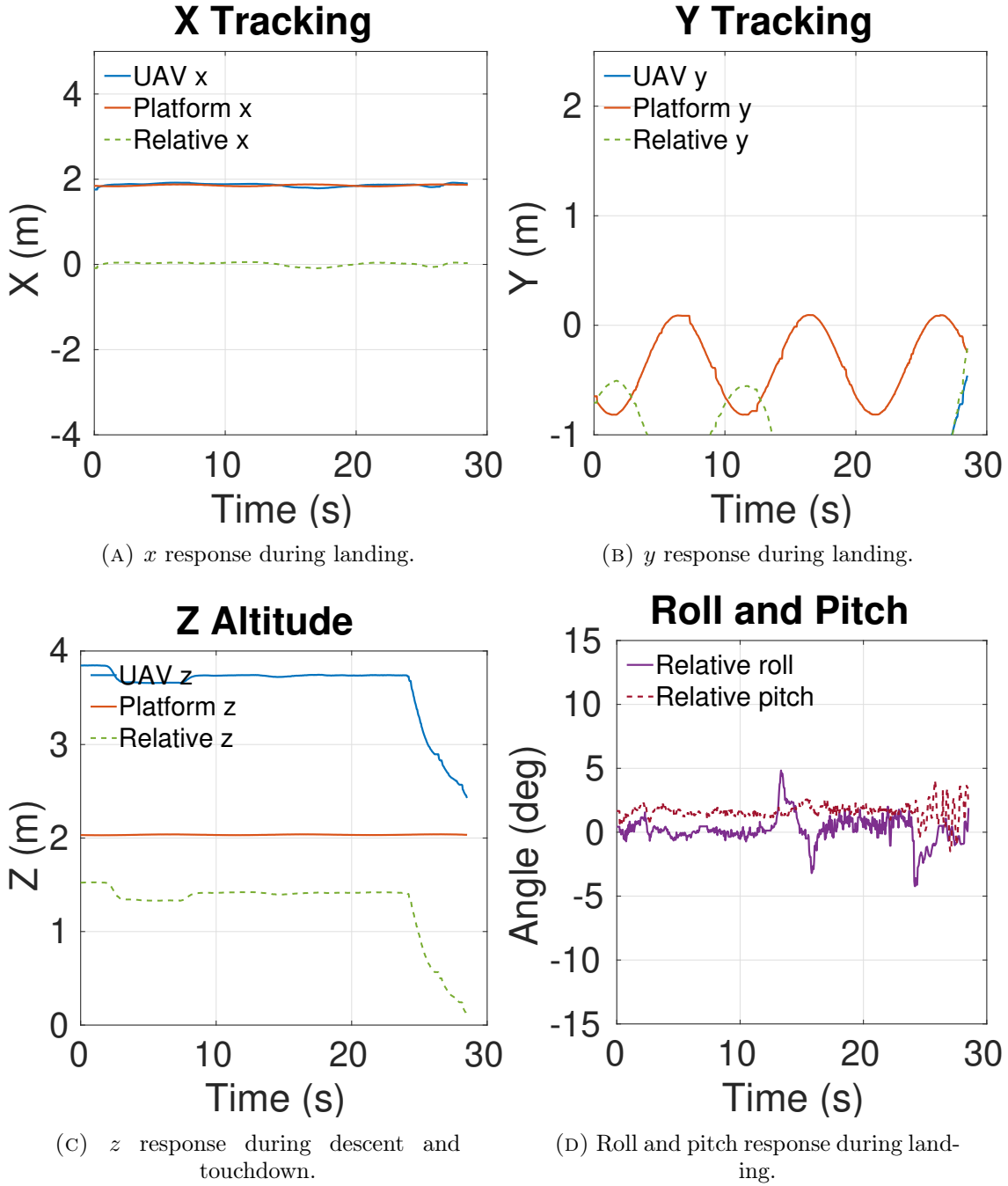


FIGURE 4.5: RL landing rollout on sinusoidal deck motion with amplitude 0.45 m and frequency 0.1 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 25$ s.

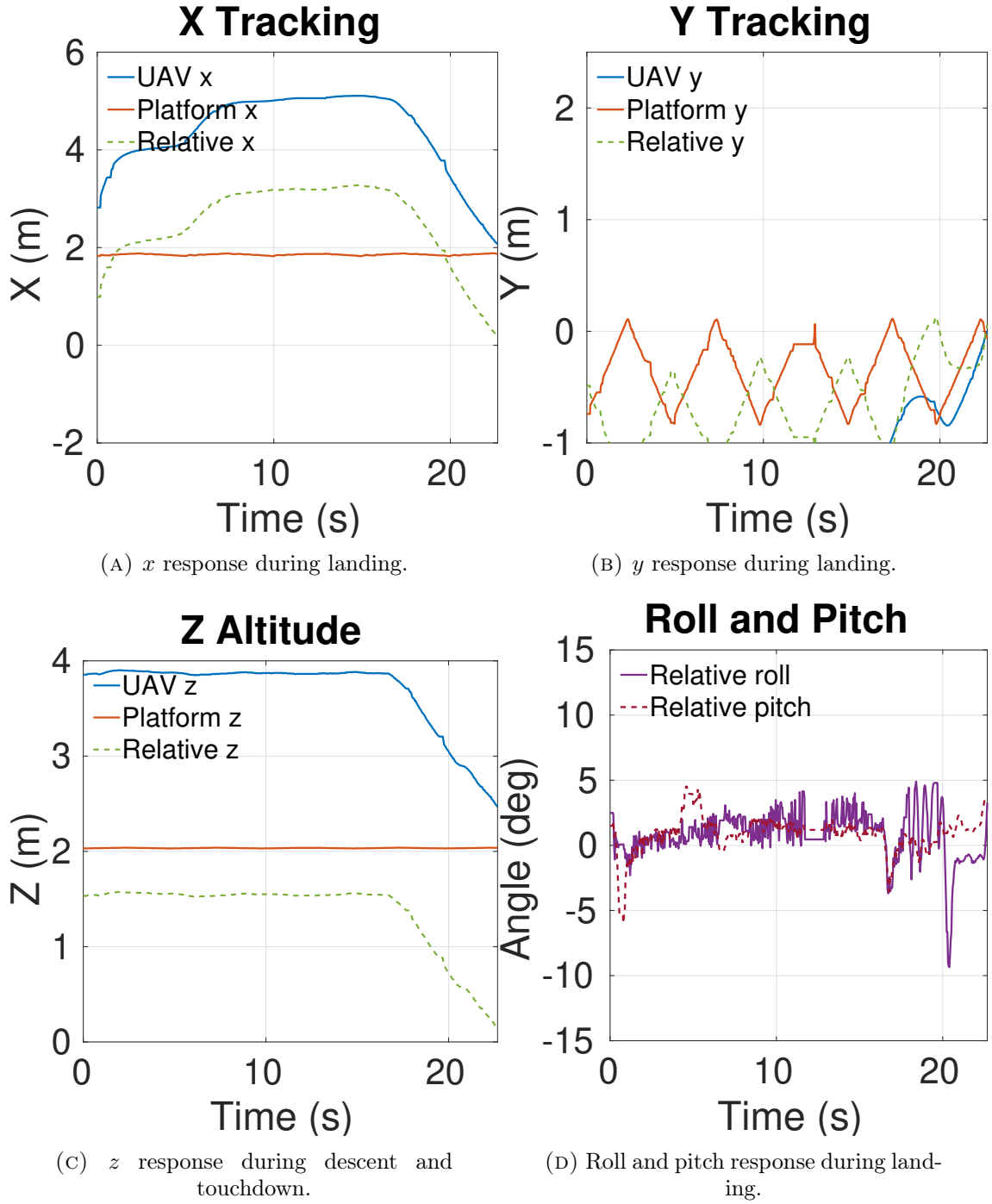


FIGURE 4.6: RL landing rollout on sinusoidal deck motion with amplitude 0.45 m and frequency 0.2 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 17$ s.

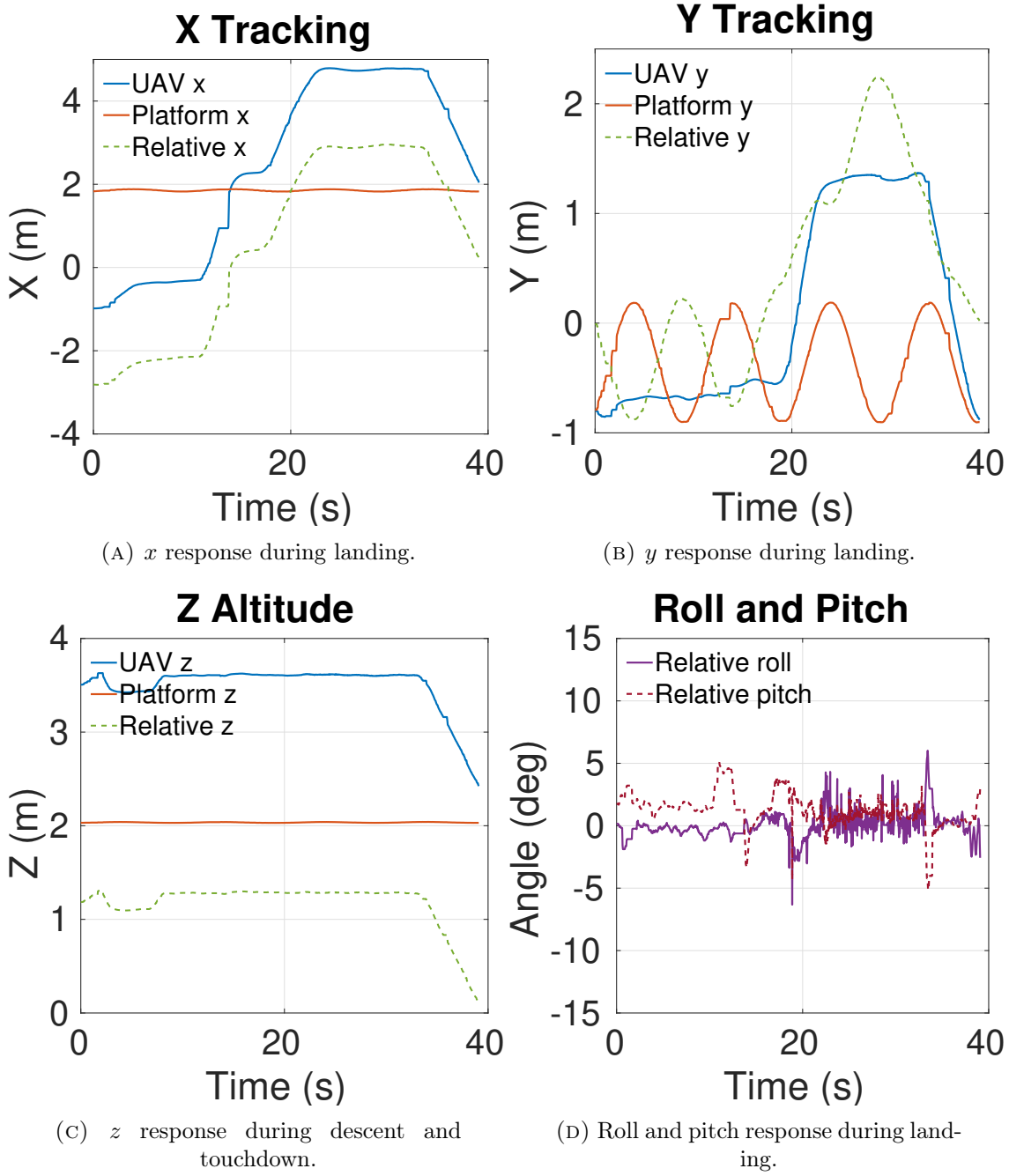


FIGURE 4.7: RL landing rollout on sinusoidal deck motion with amplitude 0.55 m and frequency 0.1 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 33$ s.

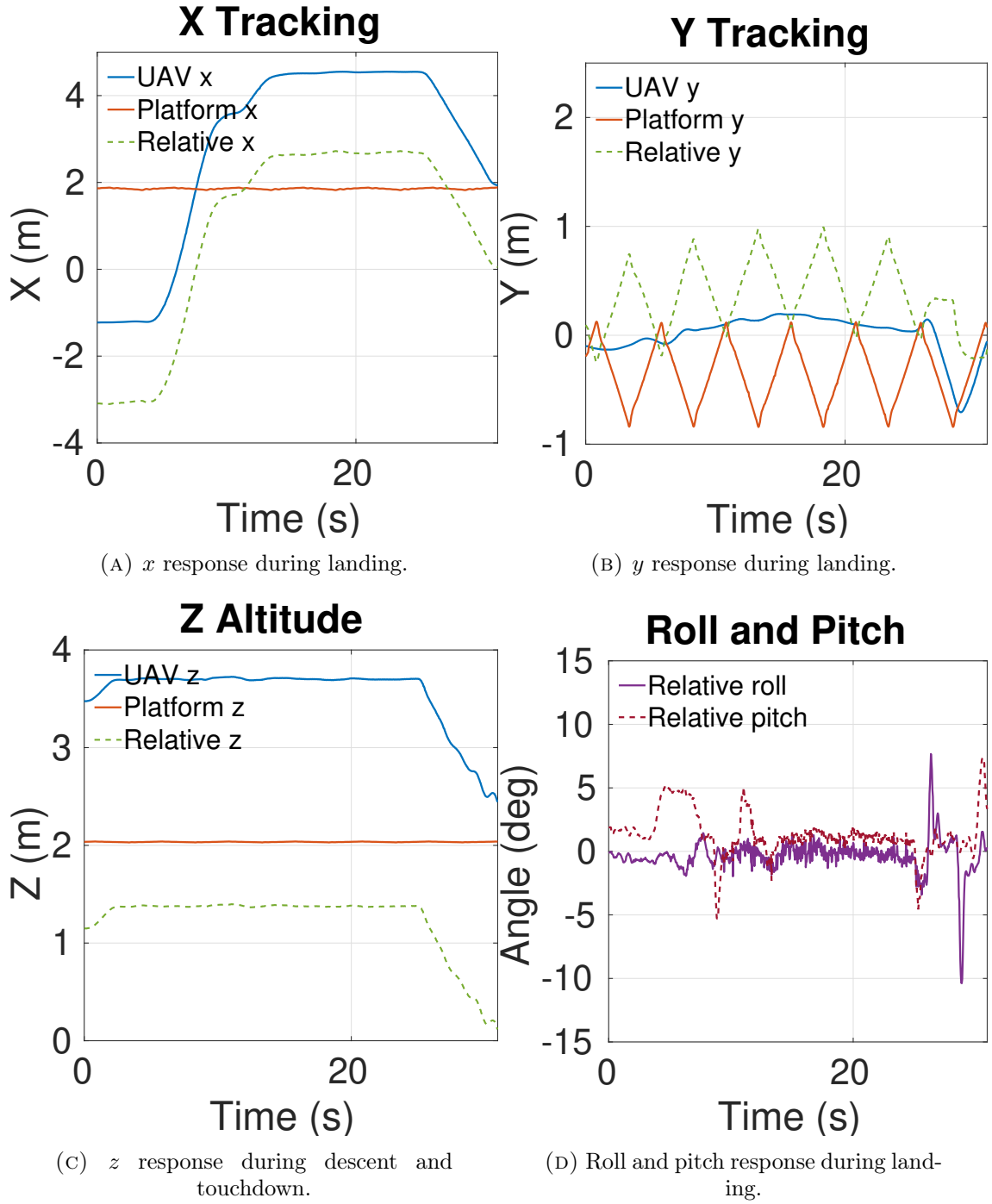


FIGURE 4.8: RL landing rollout on sinusoidal deck motion with amplitude 0.55 m and frequency 0.2 Hz. The vehicle is manually brought to a random initial position before the policy is activated at $t = 24$ s.



(A) Policy inferencing started

(B) UAV approaches platform



(C) Agile manoeuvres for precision

(D) UAV landed on the moving platform

FIGURE 4.9: RL landing snapshots on sinusoidal deck motion with amplitude 0.55 m and frequency 0.2 Hz.

References

- [1] A. Gautam, P. Sujit, and S. Saripalli, “A survey of autonomous landing techniques for uavs,” in *2014 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, May 2014, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1109/icuas.2014.6842377>
- [2] L. Xin, Z. Tang, W. Gai, and H. Liu, “Vision-based autonomous landing for the uav: A review,” *Aerospace*, vol. 9, no. 11, p. 634, Oct. 2022, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.3390/aerospace9110634>
- [3] G. Xu, Y. Zhang, S. Ji, Y. Cheng, and Y. Tian, “Research on computer vision-based for uav autonomous landing on a ship,” *Pattern Recognition Letters*, vol. 30, no. 6, p. 600–605, Apr. 2009, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1016/j.patrec.2008.12.011>
- [4] Z. Fucen, S. Haiqing, and W. Hong, “The object recognition and adaptive threshold selection in the vision system for landing an unmanned aerial vehicle,” in *2009 International Conference on Information and Automation*. IEEE, Jun. 2009, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1109/icinfa.2009.5204904>

- [5] A. Benini, M. J. Rutherford, and K. P. Valavanis, “Real-time, gpu-based pose estimation of a uav for autonomous takeoff and landing,” in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 3463–3470.
- [6] M. Fiala, “Comparing artag and artoolkit plus fiducial marker systems,” in *IEEE International Worksho on Haptic Audio Visual Environments and their Applications, 2005*. IEEE, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1109/have.2005.1545669>
- [7] X. Zhang, J. Jiang, Y. Fang, X. Zhang, and X. Chen, “Enhanced fiducial marker based precise landing for quadrotors,” in *2019 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*. IEEE, Jul. 2019, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1109/aim.2019.8868532>
- [8] J. Wang and E. Olson, “Apriltag 2: Efficient and robust fiducial detection,” in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2016, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1109/iros.2016.7759617>
- [9] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer, “Speeded up detection of squared fiducial markers,” *Image and Vision Computing*, vol. 76, p. 38–47, Aug. 2018, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1016/j.imavis.2018.05.004>
- [10] M. Bhargavapuri, A. K. Shastri, H. Sinha, S. R. Sahoo, and M. Kothari, “Vision-based autonomous tracking and landing of a fully-actuated rotorcraft,” *Control Engineering Practice*, vol. 89, p. 113–129, Aug. 2019, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1016/j.conengprac.2019.05.015>

-
- [11] E. Kawamura, K. Kannan, T. Lombaerts, and C. A. Ippolito, “Vision-based precision approach and landing for advanced air mobility,” in *AIAA SCITECH 2022 Forum*. American Institute of Aeronautics and Astronautics, Jan. 2022, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.2514/6.2022-0497>
- [12] T. K. Venugopalan, T. Taher, and G. Barbastathis, “Autonomous landing of an unmanned aerial vehicle on an autonomous marine vehicle,” in *2012 Oceans*. IEEE, Oct. 2012, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1109/oceans.2012.6404893>
- [13] S. Lin, M. A. Garratt, and A. J. Lambert, “Monocular vision-based real-time target recognition and tracking for autonomously landing an uav in a cluttered shipboard environment,” *Autonomous Robots*, vol. 41, no. 4, p. 881–901, Apr. 2016, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1007/s10514-016-9564-2>
- [14] L. Wang and X. Bai, “Quadrotor autonomous approaching and landing on a vessel deck,” *Journal of Intelligent & Robotic Systems*, vol. 92, no. 1, p. 125–143, Dec. 2017, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1007/s10846-017-0757-5>
- [15] J. L. Sanchez-Lopez, J. Pestana, S. Saripalli, and P. Campoy, “An approach toward visual autonomous ship board landing of a vtol uav,” *Journal of Intelligent & Robotic Systems*, vol. 74, no. 1–2, p. 113–127, Oct. 2013, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.1007/s10846-013-9926-3>
- [16] R. Polvara, S. Sharma, J. Wan, A. Manning, and R. Sutton, “Vision-based autonomous landing of a quadrotor on the perturbed deck of an unmanned

- surface vehicle,” *Drones*, vol. 2, no. 2, p. 15, Apr. 2018, (accessed May 18, 2026). [Online]. Available: <http://dx.doi.org/10.3390/drones2020015>
- [17] A. Shastry, A. Datta, and I. Chopra, “Vision-based autonomous uas landing on a stochastically moving platform,” 05 2021.
- [18] B. Lee, V. Saj, D. Kalathil, and M. Benedict, “Intelligent vision-based autonomous ship landing of vtol uavs,” *Journal of the American Helicopter Society*, vol. 68, no. 2, pp. 113–126, 2023.
- [19] C. Prachand, R. Rustagi, R. Shankar, J. Singh, A. Abhishek, and K. Venkatesh, “Vision-based autonomous ship deck landing of an unmanned aerial vehicle using fractal aruco markers,” *AIAA SCITECH 2025 Forum*, 2025.
- [20] P. M. Gupta, É. Pairet, T. Nascimento, and M. Saska, “Landing a UAV in harsh winds and turbulent open waters,” *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 744–751, 2023.
- [21] O. Procházka, F. Novák, T. Báča, P. M. Gupta, R. Pěnička, and M. Saska, “Model predictive control-based trajectory generation for agile landing of unmanned aerial vehicle on a moving boat,” *Ocean Engineering*, vol. 313, p. 119164, 2024.
- [22] B. Hu, L. Lu, and S. Mishra, “A control architecture for time-optimal landing of a quadrotor onto a moving platform,” *Asian Journal of Control*, vol. 20, no. 5, pp. 1701–1712, 2018.
- [23] N. Xuan-Mung, S. K. Hong, N. P. Nguyen, L. N. N. T. Ha, and T.-L. Le, “Autonomous quadcopter precision landing onto a heaving platform: New method and experiment,” *IEEE Access*, vol. 8, pp. 167 192–167 207, 2020.

- [24] M. Triantafyllou, M. Bodson, and M. Athans, “Real time estimation of ship motions using kalman filtering techniques,” *IEEE Journal of Oceanic Engineering*, vol. OE-8, no. 1, pp. 9–20, 1983.
- [25] M. Richter, K. Schneider, D. Walser, and O. Sawodny, “Real-time heave motion estimation using adaptive filtering techniques,” in *19th IFAC World Congress*, 2014, pp. 10 119–10 125.
- [26] M. Sidar and B. F. Doolin, “On the feasibility of real-time prediction of aircraft carrier motion at sea,” NASA Ames Research Center, Tech. Rep. NASA TM X-62454, 1975.
- [27] J. Keijzer, “Predicting ship heave motions: Ship heave motion prediction using LSTM neural networks without wave excitation information,” Master’s thesis, Utrecht University, 2022.
- [28] X. Geng, Y. Li, and Q. Sun, “A novel short-term ship motion prediction algorithm based on EMD and adaptive PSO–LSTM with the sliding window approach,” *Journal of Marine Science and Engineering*, vol. 11, no. 3, p. 466, 2023.
- [29] Y. Yang, T. Peng, S. Liao, and J. Li, “Real-time prediction of ship motions based on the reservoir computing model,” *Journal of Ocean Engineering and Science*, 2025.
- [30] S. Herrero, C. Pinto, O. Altuzarra, and M. Diez, “Analysis of the 2 p ru-1 p rs 3dof parallel manipulator: kinematics, singularities and dynamics,” *Robotics and Computer-Integrated Manufacturing*, vol. 51, pp. 63–72, 06 2018.
- [31] S. Staicu, “Dynamic analysis of the 3-3 stewart platform,” *UPB Scientific Bulletin D*, vol. 71, no. 2, pp. 3–18, 2009.

- [32] M. H. Abedinnasab, Y.-J. Yoon, and H. Zohoor, “Exploiting higher kinematic performance-using a 4-legged redundant pm rather than gough-stewart platforms,” *Serial and Parallel Robot Manipulators-Kinematics, Dynamics, Control and Optimization*, vol. 10, p. 32141, 2012.
- [33] R. Shankar, C. Prachand, J. Singh, Abhishek, and K. S. Venkatesh, “Vision-based landing of uav on simulated ship deck with roll, pitch and sway motions,” in *Proceedings of the Vertical Flight Society Forum*, vol. 81. Vertical Flight Society, 2025, (accessed May 18, 2026). [Online]. Available: <https://proceedings.vtol.org/81/autonomy-and-uas/vision-based-landing-of-uav-on-simulated-ship-deck-with-roll--pitch-and-sway-motions>
- [34] C. Hendrick, D. Nicholson, E. Jaques, J. Horn, J. Langelaan, and A. Sydney, “Scaled experiments in flight control design for autonomous landing in high sea states,” Jun. 2022.
- [35] A. Schwartz, “Systematic Characterization of the Naval Environment (SCONE) - Standard Deck Motion Data for a Generic Surface Combatant,” <http://www.navalengineers.org/Symposia/PastSymposia/Launch-and-Recovery-2016/Program/Schwartz>, 2015, (accessed May 18, 2026).
- [36] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ: Wiley, 2015.
- [37] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.

- [38] X. Zhao, R. Xu, and C. Kwan, “Ship-motion prediction: Algorithms and simulation results,” in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2004.
- [39] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [40] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [41] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, “acados: a modular open-source framework for fast embedded optimal control,” <https://arxiv.org/abs/1910.13753>, 2020, (accessed May 18, 2026).
- [42] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [43] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *2019 18th European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [44] “CBF Optimised Vision-Based UAV Landing on a Heaving Shipdeck Emulator,” https://www.youtube.com/watch?v=_4y6GI-JtU8, 2026, (accessed May 18, 2026).

- [45] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature*, vol. 620, pp. 982–987, 2023.
- [46] E. Kaufmann, L. Bauersfeld, and D. Scaramuzza, “A benchmark comparison of learned control policies for agile quadrotor flight,” <https://arxiv.org/abs/2202.10796>, 2022, (accessed May 18, 2026).
- [47] Y. Song, K. Shi, R. Penicka, and D. Scaramuzza, “Learning perception-aware agile flight in cluttered environments,” <https://arxiv.org/abs/2210.01841>, 2022, (accessed May 18, 2026).
- [48] L. Meier, D. Honegger, and M. Pollefeys, “Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms,” in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 6235–6240.
- [49] R. Shankar, “uav_rl: Px4-compatible reinforcement learning training and transfer stack for uav landing,” https://github.com/s-ritwik/uav_rl, 2026, (accessed May 18, 2026).
- [50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [51] “Reinforcement Learning based Autonomous UAV Quadcopter Landing on a moving Platform | Part 1,” <https://www.youtube.com/watch?v=pRaUoR9tWNc>, 2026, (accessed May 18, 2026).
- [52] “Reinforcement Learning based Autonomous UAV Quadcopter Landing on a moving Platform | Part 2,” <https://www.youtube.com/watch?v=cQChESdLzrM>, 2026, (accessed May 18, 2026).